CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

CURSO DE ENGENHARIA DE COMPUTAÇÃO

VICTOR DE SOUZA VILELA DA SILVA

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:

Implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web

Leopoldina

2026

A326m Silva, Victor de Souza Vilela da.

2026 Interpretador Dinâmico para Ensino de Programação : Implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web / Victor de Souza Vilela da Silva

Vilela - Leopoldina : CEFET-MG, 2026.

viii, 102 f. : il.

Orientador: Luís Augusto Mattos Mendes
Coorientador: Eduardo Gabriel Reis Miranda

TCC (Graduação) – Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina. Engenharia de Computação.

1. Interpretador dinâmico. 2. Ensino de programação. 3. Compiladores. 4. Análise léxica. 5. Código intermediário. I. Mendes, Luís Augusto Mattos. II. Miranda, Eduardo Gabriel Reis. III. Interpretador dinâmico para ensino de programação: implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web.

CDU 004.414.23

Elaboração da ficha catalográfica pela bibliotecária Luzia Adriana Damasceno, CRB 6º 2305 / CEFET-MG campus Leopoldina.

VICTOR DE SOUZA VILELA DA SILVA

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO: Implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web

Trabalho de conclusão de curso apresentado ao Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, do Campus Leopoldina, como parte do requisito para obtenção do título de Bacharel em Engenharia de Computação.

Banca Examinadora:

Prof. Me. Luís Augusto Mattos Mendes- Orientador
CEFET-MG

Prof. Dsc. Eduardo Gabriel Reis Miranda- Coorientador
CEFET-MG

Prof. Me. Matheus Ávila Moreira de Paula
CEFET-MG

Prof. Dsc. Gabriella Castro Barbosa Costa
CEFET-MG

(Assinaturas Digitais)

Leopoldina
2026

AGRADECIMENTOS

Agradeço à minha família pelo apoio incondicional, incentivo e paciência ao longo desta jornada. Sem vocês, nada disso seria possível. Meu sincero agradecimento também ao meu orientador, Luís Augusto Mattos Mendes, e ao meu coorientador, Eduardo Gabriel Reis Miranda, pela orientação, dedicação e ensinamentos valiosos que tornaram este trabalho possível. A todos que, de alguma forma, contribuíram para esta conquista, meu muito obrigado.

RESUMO

O ensino introdutório de programação envolve a aprendizagem de conceitos algorítmicos e de convenções de escrita da linguagem. O projeto do qual este trabalho faz parte propõe uma plataforma educacional composta por uma IDE web e um núcleo de tradução e execução configurável. A plataforma organiza-se em dois eixos complementares: a interface e a experiência de uso, tratadas no documento complementar, e o núcleo, objeto deste trabalho. O objetivo deste documento é projetar, implementar e integrar esse núcleo, contemplando análise léxica, sintática e semântica, geração de código intermediário e execução por um interpretador. O qualificativo dinâmico refere-se à aplicação de configurações de vocabulário e à seleção de variantes predefinidas de blocos, terminadores, tipagem e arranjos a cada novo processamento do código. O usuário não acrescenta produções gramaticais arbitrárias nem altera a semântica das operações durante uma execução. Os procedimentos compreendem revisão de literatura, modelagem da arquitetura e implementação incremental com testes automatizados. A solução permite inspecionar os tokens e as instruções intermediárias e executar programas no navegador, além de reutilizar o núcleo no servidor para a avaliação automática de exercícios. O diferencial técnico está em reunir configuração da linguagem, inspeção da tradução e execução em uma IDE. A contribuição pedagógica dessa combinação permanece como hipótese: a avaliação com estudantes não integra o escopo deste documento, e a consolidação do protocolo de avaliação é registrada como encaminhamento.

Palavras-chave: Interpretador dinâmico; Educação em Programação Introdutória; Programação Orientada à Linguagem; Ambiente de Programação Interativo; ensino-aprendizagem.

ABSTRACT

Introductory programming education involves learning both algorithmic concepts and language conventions. The broader project proposes an educational platform consisting of a web IDE and a configurable translation and execution core. The platform has two complementary scopes: the interface and user experience, addressed in the companion document, and the core, addressed in this work. This document aims to design, implement, and integrate the core, covering lexical, syntactic, and semantic analysis, intermediate code generation, and execution by an interpreter. The term dynamic refers to applying vocabulary configurations and selecting predefined variants of blocks, statement terminators, typing, and arrays for each new processing of source code. Users cannot introduce arbitrary grammar productions or change the semantics of operations during program execution. The procedures comprise a literature review, architectural modeling, and incremental implementation with automated tests. The solution supports token and intermediate instruction inspection and program execution in the browser, while also reusing the core on the server for automated exercise assessment. Its technical contribution is the combination of language configuration, translation inspection, and execution within an IDE. The pedagogical contribution of this combination remains a hypothesis: evaluation with students is outside the scope of this document, and completion of the evaluation protocol is identified as future work.

Keywords: Dynamic Interpreter. Introductory Programming Education. Language-Oriented Programming. Interactive Programming Environment. Teaching-Learning.

LISTA DE ILUSTRAÇÕES

Figura 2.1 – Arquitetura de um compilador moderno. 16
Figura 2.2 – Exemplificação de agrupamento de caracteres em lexemas. 18
Figura 2.3 – Ciclo de Desenvolvimento Guiado por Testes (TDD). 29
Figura 3.1 – Casos de uso: acesso e ambiente de desenvolvimento. 44
Figura 3.2 – Casos de uso: participação e gestão pedagógica. 45
Figura 4.1 – Arquitetura e locais de execução do núcleo na plataforma. 52
Figura 4.2 – Tabelas e referências do esquema lógico da plataforma. 53
Figura 4.3 – Funções de validação prévia da configuração léxica da linguagem. 61

LISTA DE TABELAS

Tabela 2.1 – Comparativo entre as ferramentas analisadas e a plataforma proposta, segundo os três eixos de análise. 41
Tabela 3.1 – Requisitos funcionais da plataforma. 46
Tabela 3.2 – Requisitos não funcionais da plataforma. 47
Tabela 3.3 – Cronograma de desenvolvimento do trabalho de conclusão de curso. 48
Tabela 4.1 – Elementos da linguagem passíveis de redefinição pelo usuário. 59
Tabela 4.2 – Analisadores especializados (*scanners*) despachados pela `LexerScannerFactory`. 63
Tabela 4.3 – Exemplos de seleção de dados nas respostas do back-end. 71
Tabela 4.4 – Distribuição dos testes executados no back-end. 76
Tabela 4.5 – Blocos do instrumento PA-01, procedência e número de itens. 78
Tabela B.1 – Grupos de recursos da API da plataforma. 94
Tabela C.1 – Distribuição dos casos de teste por camada do compilador. 95

SUMÁRIO

1	**INTRODUÇÃO**	**11**
1.1	Justificativa	12
1.2	Objetivos	13
1.2.1	Objetivo Geral	13
1.2.2	Objetivos Específicos	13
1.3	Estrutura do Trabalho	14
2	**REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS**	**15**
2.1	Compiladores	15
2.1.1	Compiladores e seus Fundamentos	15
2.1.2	O Modelo de Análise e Síntese (Fases da Compilação)	16
2.1.3	A Análise Léxica e o Reconhecimento de *Tokens*	18
2.1.4	Estratégias de Análise Sintática: *Top-Down* e *Bottom-Up*	19
2.1.5	Análise Semântica e Verificação de Tipos	21
2.1.6	Geração de Código Intermediário	22
2.1.7	Otimização de Código e Eficiência Computacional	24
2.1.8	Geração de Código Objeto e Mapeamento de Arquitetura	25
2.1.9	Execução de Código de Máquina	26
2.2	Engenharia de Software	27
2.2.1	Desenvolvimento Iterativo e Incremental e *Extreme Programming*	27
2.2.2	Testes Automatizados e *Test-Driven Development* (TDD)	28
2.2.3	Arquitetura Limpa e Separação de Responsabilidades	30
2.2.4	Qualidade de Código e *Clean Code*	30
2.2.5	FastAPI: Framework para APIs de Alto Desempenho	32
2.2.6	Testes de integração de APIs assíncronas	33
2.2.7	Contratos de resposta e minimização dos dados	33
2.2.8	Autorização e proteção de recursos no back-end	34
2.3	Softwares Educativos e Tecnologias de Apoio à Aprendizagem	35
2.4	Trabalhos Relacionados	35
2.4.1	Critérios de Seleção dos Trabalhos	36

2.4.2 Ambientes Voltados à Iniciação em Programação 37
2.4.3 Projetos Didáticos de Construção de Compiladores 39
2.4.4 Análise Crítica e o Diferencial da Solução Proposta 40
3 PROPOSTA 43
4 METODOLOGIA 49
4.1 Modelagem da Arquitetura do Projeto 50
4.1.1 Modelagem do Banco de Dados 51
4.1.2 Especificação das Rotas da API 55
4.2 Especificação da Linguagem e de seus Elementos Configuráveis 57
4.3 Interpretador 62
4.3.1 Análise Léxica 62
4.3.2 Análise Sintática 64
4.3.3 Análise Semântica 65
4.3.4 Geração de Código Intermediário 66
4.3.5 Interpretação e Tratamento de Erros em Tempo de Execução 67
4.4 Integração entre a Plataforma e o Interpretador 69
4.5 Contratos de Resposta e Controles do Back-end 70
4.5.1 Limites dos controles examinados 72
4.6 Estratégia de Testes Automatizados 72
4.6.1 Testes do núcleo de tradução 72
4.6.2 Testes do serviço FastAPI 74
4.6.3 Resultados e alcance da verificação 75
4.6.4 Testes no fluxo de integração contínua 76
4.7 Protocolo de Avaliação da Solução 77
4.7.1 Quem responde e o que é possível concluir 77
4.7.2 Composição do instrumento 77
4.7.3 Itens sobre o núcleo de tradução 78
4.7.4 Escalas usadas e cálculo dos escores 79
4.7.5 Participantes, procedimento e questões éticas 80
4.7.6 Limites de interpretação e continuidade 81
5 RESULTADOS DA AVALIAÇÃO 82
5.1 Composição do Painel 82

5.2 Avaliação do Núcleo de Tradução 82
5.3 Suficiência do Subconjunto da Linguagem 82
5.4 Mensagens de Erro com Vocabulário Personalizado 82
5.5 Usabilidade e Experiência 82
5.6 Potencial Pedagógico e Intenção de Uso 82
5.7 Riscos e Limitações Apontados pelo Painel 82
5.8 Respostas Abertas 82
5.9 Síntese e Confronto com os Objetivos 82
6 CONSIDERAÇÕES FINAIS 83
6.1 Retomada dos Objetivos 83
6.2 Limitações 84
6.3 Encaminhamentos 85
REFERÊNCIAS 87
A APÊNDICE A – PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES 92
A.1 Atividades Desenvolvidas em Regime Colaborativo 92
A.2 Eixos de Aprofundamento Individual 92
A.2.1 Victor de Souza Vilela da Silva (esta monografia) 92
A.2.2 Igor Lamoia Queiroz (documento complementar) 93
B APÊNDICE B – DOCUMENTAÇÃO DA API 94
C APÊNDICE C – CASOS DE TESTE AUTOMATIZADOS 95
D APÊNDICE D – DICIONÁRIO DE DADOS 96

1 INTRODUÇÃO

Este Trabalho de Conclusão de Curso (TCC) foi desenvolvido em parceria com Igor Lamoia Queiroz, aluno do Curso de Engenharia de Computação do CEFET-MG, Unidade Leopoldina. Os dois trabalhos tratam do mesmo projeto, mas com enfoques distintos, conforme a divisão de responsabilidades descrita no Apêndice A. Este texto descreve o núcleo do compilador: as fases de análise léxica, análise sintática, geração de código intermediário e execução pelo interpretador. O trabalho complementar, intitulado *Interpretador Dinâmico para Ensino de Programação: Uma abordagem interativa com Ambiente de Desenvolvimento Integrado (IDE) para personalização e execução de comandos definidos pelo usuário*, foca na IDE web, na interface de personalização da linguagem e na experiência de uso da plataforma. Por se tratar de trabalhos complementares e interdependentes, para a compreensão do sistema como um todo, recomenda-se ler os dois trabalhos em conjunto.

O ensino introdutório de programação envolve dificuldades de compreensão e aplicação dos conceitos, discutidas no mapeamento sistemático de Souza, Batista e Barbosa (2016). Neste trabalho, o recorte investigado é a exigência de lidar simultaneamente com a construção do algoritmo e com as convenções de escrita da linguagem. Os experimentos de Stefik e Siebert (2013) indicam que escolhas de sintaxe afetam o desempenho de iniciantes; esse resultado motiva investigar alternativas de apresentação da linguagem, sem pressupor que uma nova ferramenta produza, por si só, ganhos de aprendizagem.

Em linguagens textuais como C, C++ e Java, o programa precisa respeitar convenções de palavras reservadas, delimitadores e terminadores de instrução para ser aceito pelo tradutor. Um erro nessas convenções pode impedir a tradução antes da execução do algoritmo. A distinção entre a intenção do estudante e a forma aceita pela linguagem é relevante para o ensino, pois parte do esforço inicial recai sobre a identificação de erros sintáticos (AHO; SETHI; ULLMAN, 1995; STEFIK; SIEBERT, 2013).

O artefato desenvolvido combina compilação para uma representação intermediária e interpretação dessa representação. Neste texto, **compilador** designa as etapas que analisam o código-fonte e geram instruções de três endereços; **interpretador** designa o componente que executa essas instruções. A expressão **núcleo de tradução e execução** abrange os dois componentes. O sistema é apresentado como um interpretador porque a execução do programa depende desse componente, sem geração de um executável nativo para o usuário. Essa

organização é compatível com a distinção entre tradução e execução discutida por Aho, Sethi e Ullman (1995) e Nystrom (2021).

O qualificativo **dinâmico** refere-se à configuração da linguagem recebida a cada nova análise e execução: o usuário pode redefinir palavras-chave, operadores em forma de palavra, literais booleanos e delimitadores, além de selecionar modos de terminação de instruções, de blocos, de tipagem e de arranjos já implementados no núcleo. Essas escolhas não exigem recompilar a aplicação web. O termo não significa que o usuário possa acrescentar produções gramaticais arbitrárias ou alterar a semântica das operações durante a execução de um programa. Os limites dessa configuração são detalhados na Seção 4.2.

Diante disso, este trabalho parte da seguinte pergunta: **em que medida a personalização da sintaxe da linguagem pelo próprio estudante contribui para reduzir a barreira de aprendizagem enfrentada no primeiro contato com a programação?**

Trabalha-se com a hipótese, ainda não submetida a verificação empírica, de que permitir ao aluno configurar os elementos de escrita da linguagem possa favorecer a experimentação de algoritmos e reduzir dificuldades associadas à memorização dessas convenções. A hipótese diz respeito à personalização pelo estudante, inclusive quando os termos escolhidos estão em português, e não apenas à tradução de palavras de uma língua estrangeira.

Cumpre delimitar o alcance deste trabalho diante dessa pergunta. O que aqui se apresenta é a construção do artefato que torna a investigação possível e a delimitação de uma avaliação futura, cujo protocolo ainda precisa ser consolidado. A aplicação desse protocolo junto a estudantes, que permitiria responder empiricamente à pergunta formulada, não integra o escopo deste documento e é registrada como encaminhamento futuro.

1.1 Justificativa

Já existem abordagens que reduzem a exposição inicial a convenções de linguagens profissionais. O Portugol Studio utiliza comandos em português e oferece execução passo a passo e inspeção de variáveis (UNIVALI, 2026). O Scratch adota programação por blocos e disponibiliza materiais para a criação de projetos interativos (Scratch Foundation, 2026). Portanto, a dificuldade discutida neste trabalho não pode ser atribuída de forma geral ao uso de uma língua estrangeira, nem se presume que as ferramentas existentes ignorem esse problema.

O diferencial investigado é mais delimitado: permitir ao estudante configurar o

vocabulário de uma linguagem textual e selecionar variantes de escrita previstas no núcleo, mantendo observáveis os *tokens* e as instruções intermediárias. Isso difere tanto de utilizar um vocabulário em português previamente definido quanto de compor programas por blocos. A comparação restringe-se às ferramentas e aos critérios apresentados na Seção 2.4, sem pretensão de caracterizar a totalidade dos ambientes educacionais existentes.

A proposta é construir esse núcleo configurável e integrá-lo a uma IDE web para experimentação. Uma possível continuidade didática é a resolução de exercícios em linguagens textuais como C++, Java e Python, para as quais o juiz *online* Beecrowd apresenta exemplos de entrada e saída em sua documentação (beecrowd, 2026). A contribuição da personalização para essa transição é uma expectativa a ser investigada, e não um resultado demonstrado neste documento.

1.2 Objetivos

A seguir são apresentados o objetivo geral e os objetivos específicos que nortearam o desenvolvimento desta pesquisa e a construção da ferramenta.

1.2.1 Objetivo Geral

Projetar e desenvolver um interpretador dinâmico integrado a uma IDE web que permita ao usuário personalizar a sintaxe e o vocabulário da linguagem, com o objetivo de facilitar o processo de ensino-aprendizagem de algoritmos para estudantes iniciantes.

1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Especificar a gramática da linguagem e os elementos passíveis de redefinição pelo usuário em tempo de execução;
- Projetar os módulos de análise léxica, sintática e semântica com suporte à configuração de vocabulário e à seleção das variantes de linguagem previstas na implementação;
- Implementar os módulos projetados, integrando-os em um interpretador dinâmico executável no navegador;

- Integrar o núcleo do interpretador à interface web, definindo os contratos de comunicação entre os módulos;
- Disponibilizar o núcleo de tradução em ambiente web, verificando seu comportamento por meio de uma suíte de testes automatizados que cubra as construções da linguagem;
- Definir o protocolo de avaliação da solução, estabelecendo os critérios que permitirão, em etapa posterior, aferir sua contribuição pedagógica.

1.3 Estrutura do Trabalho

Este documento está organizado da seguinte forma: o Capítulo 2 apresenta o referencial teórico e os trabalhos relacionados, incluindo os critérios de seleção das ferramentas. O Capítulo 3 descreve o escopo e o planejamento do trabalho. O Capítulo 4 detalha a arquitetura e a implementação do núcleo de tradução e execução, a persistência de dados, a integração com a plataforma web e a estratégia de testes. O Capítulo 6 retoma os objetivos, as limitações e os encaminhamentos. Os apêndices complementam a descrição com a divisão de responsabilidades, os recursos da API, os testes e o dicionário de dados.

2 REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS

Este capítulo reúne a fundamentação teórica que sustenta o trabalho. Inicialmente, a Seção 2.1 apresenta os fundamentos de compiladores e suas fases, da análise léxica à geração de código objeto e à execução do código de máquina. A Seção 2.2 trata dos conceitos de engenharia de software que orientaram a construção do sistema: metodologias ágeis, testes automatizados, arquitetura em camadas, qualidade de código e o *framework* adotado no *back-end*. Em seguida, discutem-se os softwares educativos e as tecnologias de apoio à aprendizagem de programação. Por fim, a Seção 2.4 analisa os trabalhos relacionados, posicionando a solução proposta frente ao estado da arte.

2.1 Compiladores

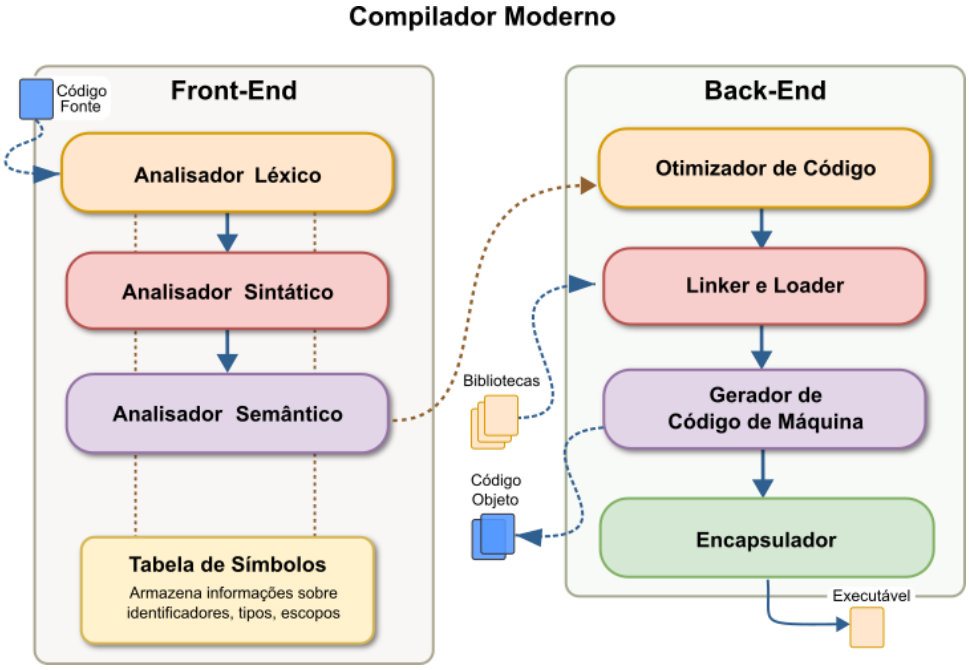
Esta seção percorre os fundamentos de compiladores que embasam o núcleo de tradução implementado neste trabalho, desde a definição de compilador e de suas fases até a geração de código intermediário.

2.1.1 Compiladores e seus Fundamentos

Posto de forma simples, um compilador é um programa de computador que lê um programa escrito em uma linguagem fonte e o traduz para um programa equivalente em uma linguagem alvo. O modelo de compilação moderno é estruturado dividindo esse processo em duas partes fundamentais: a análise e a síntese. A fase de análise, frequentemente associada ao *Front-End*, divide o programa fonte em suas partes constituintes e cria uma representação intermediária do mesmo. Por outro lado, a fase de síntese, ou *Back-End*, constrói o programa alvo desejado a partir dessa representação intermediária gerada (AHO; SETHI; ULLMAN, 1995).

A Figura 2.1 ilustra essa separação lógica e a progressão da transformação do código.

Figura 2.1 – Arquitetura de um compilador moderno.



Fonte: Alcantara (2024).

Conceitualmente, um compilador opera em fases sequenciais, cada uma das quais transforma o programa fonte de uma representação abstrata para outra. As primeiras fases formam o núcleo de análise e englobam a análise léxica, a análise sintática e a análise semântica. A partir daí, o fluxo avança para as etapas de geração de código intermediário, otimização e geração de código alvo.

Interagindo com todas essas seis fases de compilação, encontra-se o gerenciamento da tabela de símbolos. Essa estrutura de dados é essencial para o compilador, pois atua registrando todos os identificadores usados no programa e coletando informações cruciais sobre seus atributos, permitindo rápida recuperação de escopos e validação semântica ao longo de todo o processo de tradução (AHO; SETHI; ULLMAN, 1995).

2.1.2 O Modelo de Análise e Síntese (Fases da Compilação)

O processo de compilação é tradicionalmente dividido em duas partes principais: a análise e a síntese. A fase de análise, frequentemente chamada de *front-end*, concentra-se em compreender o programa fonte dividindo o código em seus elementos constituintes para criar uma representação intermediária abstrata. Em contrapartida, a fase de síntese, ou *back-end*,

foca em mapear e construir o programa alvo desejado a partir dessa representação intermediária. A separação entre *front-end* e *back-end* atua como uma interface que facilita a adaptação e o suporte a múltiplas linguagens e arquiteturas (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Para dominar a complexidade da tradução de uma linguagem de alto nível, o fluxo geral é subdividido em módulos lógicos menores, conhecidos como fases, onde cada uma transforma a representação do programa para a etapa seguinte. Em uma implementação prática, as atividades lógicas dessas fases podem ser agrupadas em "passagens" (*passes*), otimizando o uso de memória durante o processo. As principais fases que compõem o núcleo de um compilador são:

- **Análise Léxica (*Scanner*):** Constitui a primeira fase da compilação, fazendo a leitura do programa fonte caractere por caractere. Sua função é traduzir o texto para uma sequência de agrupamentos lógicos chamados de *tokens*, ao mesmo tempo em que descarta caracteres não significativos, como espaços em branco e comentários.

- **Análise Sintática (*Parser*):** Recebe a cadeia de *tokens* do analisador léxico e verifica se a estrutura gramatical do programa está correta. O *parser* agrupa os *tokens* hierarquicamente, produzindo uma estrutura de dados conhecida como árvore de derivação ou árvore de sintaxe, seja através de estratégias *top-down* (descendentes) ou *bottom-up* (redutivas).

- **Análise Semântica:** Vai além da verificação estrutural sintática, realizando checagens para garantir que os componentes do programa fazem sentido e combinam-se de forma válida durante a execução. Um componente crítico desta fase é a verificação de tipos (*type checking*), que assegura a consistência lógica do uso de variáveis e operadores.

- **Geração de Código Intermediário:** Após as análises, o compilador gera uma representação intermediária explícita que abstrai os detalhes específicos da máquina alvo, sendo o "código de três endereços" um de seus formatos clássicos. Essa etapa simplifica a implementação geral quebrando a barreira de complexidade entre a linguagem de alto nível e as instruções de baixo nível.

- **Otimização de Código:** Atua sobre o código intermediário com o objetivo de reescrevê-lo e melhorá-lo. Essa fase foca em reduzir o tempo global de execução e minimizar o espaço de memória consumido pelo programa compilado.

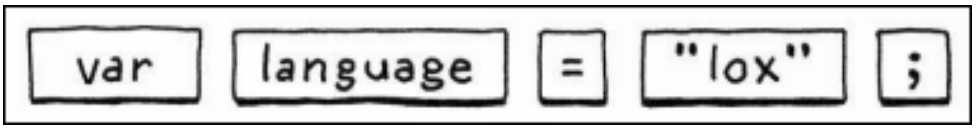
- **Geração de Código Objeto:** É a fase final de síntese, responsável por mapear o código intermediário para a linguagem alvo (código de máquina absoluto, relocável ou *assembly*). Ela exige uma seleção cuidadosa das instruções nativas da máquina, das posições de memória e a alocação eficiente dos registradores.

Ao longo de todas as fases, o compilador interage ininterruptamente com dois módulos de suporte vitais: o gerenciador da **Tabela de Símbolos** e o **Atendimento a Erros**. A tabela de símbolos registra os identificadores encontrados e armazena atributos essenciais (como escopo, tipo e métodos de transmissão de parâmetros) de maneira que permita uma recuperação extremamente rápida nas etapas posteriores. Já o módulo de erros atua detectando falhas léxicas, sintáticas ou lógicas em todas as fases, utilizando mecanismos de recuperação que re-sincronizam o analisador e permitem que a compilação prossiga na tentativa de descobrir falhas adicionais no código fonte (AHO; SETHI; ULLMAN, 1995).

2.1.3 A Análise Léxica e o Reconhecimento de *Tokens*

A análise léxica, frequentemente denominada *scanner*, constitui a primeira fase do processo de compilação. A função primária deste módulo é realizar a leitura do programa fonte, caractere por caractere, e agrupá-los logicamente em unidades com significado gramatical, denominadas lexemas, conforme exemplificado na Figura 2.2.

Figura 2.2 – Exemplificação de agrupamento de caracteres em lexemas.



Fonte: Nystrom (2021, p. 43).

Durante essa varredura, o analisador léxico também atua filtrando informações que não possuem relevância para as fases posteriores, descartando ativamente espaços em branco, tabulações e comentários inseridos pelo programador.

Neste contexto, é imprescindível estabelecer a distinção teórica entre dois conceitos fundamentais: o lexema e o *token*. Um lexema é a sequência exata de caracteres no código-fonte que representa uma unidade léxica, ou seja, é a *string* literal que foi digitada pelo usuário, como por exemplo a sequência “123” ou “var”. Por outro lado, o *token* é a representação lógica ou estrutura de dados gerada a partir desse lexema. Ele encapsula a classificação da

unidade (sua classe ou categoria sintática, como identificador, palavra reservada ou operador) e, frequentemente, o valor original do lexema e sua posição no texto (linha e coluna) para auxiliar na emissão de erros sintáticos e semânticos (NYSTROM, 2021; COOPER; TORCZON, 2012).

Para que o analisador léxico consiga identificar e classificar esses lexemas corretamente, a micro-sintaxe da linguagem de programação precisa ser formalmente descrita. Isso é realizado através de expressões regulares, que fornecem uma notação matemática concisa e poderosa para especificar a estrutura exata de cada tipo de *token* (como, por exemplo, a regra que define que um identificador deve começar com uma letra seguida de letras ou dígitos). As expressões regulares descrevem conjuntos de cadeias de caracteres chamados de linguagens regulares, que figuram entre as linguagens mais simples na hierarquia das linguagens formais (PRICE; TOSCANI, 2001).

Embora as expressões regulares sejam excelentes para a especificação humana, o reconhecimento mecânico em tempo de execução exige um formalismo executável. Para a implementação computacional, as expressões regulares são traduzidas para autômatos finitos. Um autômato finito é uma máquina abstrata de transição de estados que atua como um reconhecedor. Ele processa a fita de entrada caractere por caractere, mudando de estado lógico; se, ao final da leitura de um lexema, o autômato atingir um estado de aceitação (estado final), o *token* é validado e encaminhado para o analisador sintático.

O uso de autômatos finitos permite que a análise léxica seja extremamente eficiente, com custo de tempo linear em relação ao tamanho do código-fonte. Na prática, geradores de analisadores léxicos, como a clássica ferramenta *Lex*, recebem as expressões regulares definidas pelo projetista da linguagem e geram automaticamente o código fonte de um autômato finito determinístico capaz de reconhecer os *tokens* da linguagem de forma otimizada (COOPER; TORCZON, 2012).

2.1.4 Estratégias de Análise Sintática: *Top-Down* e *Bottom-Up*

A fase de análise sintática (ou *parsing*) tem a responsabilidade central de determinar se a cadeia de *tokens* fornecida pelo analisador léxico pode ser validamente gerada pela gramática formal da linguagem fonte. Para realizar essa verificação e construir a árvore de derivação, os compiladores adotam abordagens matemáticas que se enquadram majoritariamente em duas categorias fundamentais de algoritmos: a análise descendente (*top-down*) e a análise ascendente

ou redutiva (*bottom-up*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Análise Sintática Descendente (*Top-Down*)

A estratégia descendente consiste na tentativa de construir a árvore de derivação a partir da sua raiz (o símbolo inicial da gramática) em direção às folhas (os *tokens* terminais da cadeia de entrada). Em cada passo do processo, o analisador substitui o símbolo não-terminal mais à esquerda por um de seus lados direitos correspondentes nas regras de produção, realizando o que se denomina rigorosamente como uma derivação mais à esquerda (*leftmost derivation*).

Os analisadores *top-down* dividem-se tradicionalmente em preditivos tabulares (como o algoritmo LL(1)) e recursivos descendentes. A técnica de descida recursiva é amplamente utilizada em compiladores de produção modernos em virtude de sua elegância, facilidade de codificação manual e excelente capacidade de detectar e relatar erros de sintaxe. Contudo, para que um analisador descendente funcione de maneira eficiente e evite o custoso retrocesso (*backtracking*), a gramática subjacente da linguagem deve atender a restrições estritas. Especificamente, a gramática não pode conter recursividade à esquerda e deve estar fatorada à esquerda, de modo que o próximo *token* da entrada seja suficiente para guiar univocamente o analisador na escolha da produção correta (NYSTROM, 2021; PRICE; TOSCANI, 2001).

Análise Sintática Ascendente ou Redutiva (*Bottom-Up*)

Em contrapartida à técnica anterior, a análise ascendente processa a estrutura do programa de baixo para cima, iniciando a construção da árvore a partir das folhas (os *tokens* fornecidos pelo *scanner*) e trabalhando estruturalmente em direção à raiz.

A denominação "redutiva" deve-se à natureza do processo: o analisador busca identificar uma sequência de símbolos na pilha que corresponda exatamente ao lado direito de uma regra de produção (o *handle*) e a substitui (reduz) pelo símbolo não-terminal correspondente ao lado esquerdo daquela regra. Esse mecanismo iterativo constrói sistematicamente uma derivação mais à direita ao reverso.

A família mais proeminente e poderosa de analisadores *bottom-up* é a dos analisadores da classe LR (como SLR, LALR e o Canônico), que operam através de uma arquitetura do tipo empilha-reduz (*shift-reduce*) orientada por tabelas de transição de estados. Ao contrário da abordagem descendente, os analisadores *bottom-up* não impõem a restrição de ausência de

recursão à esquerda, acomodando e processando com eficiência tanto a recursividade à direita quanto à esquerda. Ademais, são capazes de reconhecer virtualmente todas as construções sintáticas que podem ser descritas por gramáticas livres de contexto (AHO; SETHI; ULLMAN, 1995).

Entretanto, devido à enorme complexidade matemática requerida para a construção manual das tabelas de transição de estados LR, esses analisadores não costumam ser codificados à mão; ao invés disso, recorre-se a geradores automáticos de analisadores sintáticos, como a clássica ferramenta YACC, que produzem o código-fonte do *parser* a partir de uma especificação gramatical (COOPER; TORCZON, 2012).

2.1.5 Análise Semântica e Verificação de Tipos

Esta seção apresenta os fundamentos da análise semântica e da verificação de tipos, consolidando os conceitos e definições propostos por Aho, Sethi e Ullman (1995), Cooper e Torczon (2012), Price e Toscani (2001) e Nystrom (2021).

Uma vez que a estrutura gramatical do código é validada pela análise sintática, o processo de compilação evolui para a fase semântica. O objetivo central aqui é garantir que as construções do programa, embora sintaticamente corretas, possuam coerência lógica para a futura execução. Enquanto o *parser* se limita a verificar se a "forma" do código respeita as regras da linguagem, o analisador semântico atua como um inspetor que examina o significado das combinações entre os elementos do programa. Um dos pilares desse estágio é a resolução de nomes (ou amarrações), procedimento no qual o compilador mapeia cada identificador à sua declaração correspondente, respeitando as regras de escopo (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A verificação de tipos constitui outro componente vital da análise semântica. Utilizando a árvore de derivação e as informações armazenadas na tabela de símbolos, o compilador checa se os operadores estão sendo aplicados a operandos compatíveis. Durante essa inspeção, o sistema infere os tipos das expressões e detecta interpretações de valores que seriam semanticamente inválidas. Caso a linguagem permita, o analisador também pode aplicar coerções, que são conversões implícitas de tipos (como transformar um inteiro em ponto flutuante) para viabilizar operações entre tipos distintos (PRICE; TOSCANI, 2001; NYSTROM, 2021).

Tipos de Erros Semânticos

Diferente dos erros de sintaxe, as falhas semânticas ocorrem em códigos que possuem uma estrutura gramatical perfeita, mas que violam as regras de contexto ou significado da linguagem. O foco do tratamento de erros nesta fase é capturar inconsistências lógicas, destacando-se:

- **Incompatibilidade de Tipos:** Manifesta-se quando um operador é utilizado com operandos cujos tipos não podem interagir entre si, seja por falta de suporte da linguagem ou por ausência de regras de conversão. Um caso típico em linguagens fortemente tipadas é a tentativa de realizar operações aritméticas entre textos (*strings*) e valores lógicos (booleanos).
- **Identificadores Não Declarados e Violações de Escopo:** Este erro surge quando o programa tenta acessar uma variável, função ou objeto que não foi definido ou que não está visível no bloco de código atual. Um exemplo comum é a tentativa de utilizar uma variável local fora dos limites do seu escopo de definição.
- **Inconsistências de Aridade e Estruturais:** Refere-se a falhas na conformidade numérica de elementos. Isso inclui chamadas de funções com um número de parâmetros diferente do que foi estabelecido na assinatura original, ou o acesso a um *array* utilizando um número de dimensões que não condiz com sua declaração (AHO; SETHI; ULLMAN, 1995).

2.1.6 Geração de Código Intermediário

A geração de código intermediário atua como um elo estrutural entre as fases de análise (*front-end*) e síntese (*back-end*) de um compilador. Após a verificação estrutural e semântica do código fonte, o compilador normalmente gera uma representação intermediária explícita, cujo formato é projetado para ser largamente independente das especificidades da linguagem fonte e da arquitetura da máquina alvo. A principal vantagem dessa estratégia em múltiplos passos é resolver, de maneira gradativa, a barreira de complexidade existente na conversão direta de comandos de alto nível para instruções de baixo nível. Ademais, essa etapa viabiliza a otimização de código e facilita a portabilidade, permitindo que a mesma representação interna seja traduzida para diversas arquiteturas diferentes (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Dentre as formas de representação linear utilizadas na literatura, destaca-se o uso do código de três endereços, que se assemelha a uma linguagem de montagem (*assembly*) genérica e abstrata. Nesta notação, as expressões matemáticas e lógicas são desmembradas em instruções elementares que possuem, no máximo, três operandos. A grande distinção teórica entre o código intermediário e o código objeto final reside no fato de que o intermediário se abstém de especificar detalhes físicos de *hardware*, como os registradores exatos e as posições absolutas de memória a serem alocadas (NYSTROM, 2021).

Para ilustrar o mecanismo, uma instrução aritmética convencional de atribuição, tal como `A := X + Y * Z`, seria mapeada em instruções *pseudo-assembly* mais simples, por meio da criação automática de variáveis temporárias pelo compilador. O fluxo de código de três endereços resultante desta operação seria composto por passos sequenciais lógicos:

1. `T1 := Y * Z`
2. `T2 := X + T1`
3. `A := T2`

Método de Geração Dirigida pela Sintaxe

Para que essa conversão ocorra de forma automatizada e correta, os compiladores comumente empregam a técnica denominada **tradução dirigida pela sintaxe**. Esse método estende os conceitos matemáticos das gramáticas livres de contexto, permitindo a associação direta de atributos aos símbolos gramaticais e a vinculação de ações semânticas específicas às regras de produção.

Na prática, isso significa que a geração de código pode ocorrer concomitantemente com a fase de análise sintática. À medida que o analisador (*parser*) agrupa as unidades léxicas e reconhece uma construção gramatical válida, ele executa em paralelo a ação semântica correspondente a essa regra. Essa execução processa os atributos passados pelos nós filhos da árvore gramatical e sintetiza as informações, permitindo a emissão gradativa e ordenada dos trechos de código intermediário (COOPER; TORCZON, 2012).

2.1.7 Otimização de Código e Eficiência Computacional

Situada geralmente como a etapa intermediária do processo de compilação, a otimização visa refinar a representação interna do programa para torná-la mais performática. Esse aprimoramento busca não apenas acelerar a execução e reduzir o consumo de memória, mas também, em contextos tecnológicos modernos, diminuir o gasto energético do hardware. Durante esse estágio, o compilador substitui instruções genéricas por variantes mais ágeis e semanticamente equivalentes. Um exemplo clássico é o dobramento de constantes (*constant folding*), que antecipa cálculos de valores fixos para o tempo de compilação, evitando que sejam processados repetidamente durante a execução (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para que uma técnica de otimização seja considerada válida, ela deve respeitar dois pilares fundamentais: a integridade (segurança) e a utilidade (rentabilidade). O princípio da segurança exige que qualquer modificação preserve o comportamento original do algoritmo, garantindo que o resultado final do programa não seja alterado. Já o princípio da rentabilidade assegura que a mudança estrutural resulte em um ganho de desempenho real e mensurável, justificando o tempo adicional despendido pelo compilador para realizar tal transformação (PRICE; TOSCANI, 2001).

Quanto à sua abrangência, o processo de otimização opera em diferentes níveis de profundidade, sendo categorizado em quatro escopos principais:

- **Otimização Local:** Concentra-se na análise de blocos básicos, que são sequências de instruções sem desvios internos. Nesse nível, é comum o uso de Grafos Acíclicos Dirigidos (GADs) para identificar e remover subexpressões redundantes e otimizar o uso de variáveis locais.
- **Otimização Regional:** Estende a análise para grupos de blocos básicos, focando prioritariamente em laços de repetição (*loops*). Como a maior parte do tempo de execução de um software costuma ocorrer dentro desses ciclos, melhorias aplicadas aqui geram impactos significativos no desempenho global.
- **Otimização Global (Intraprocedural):** Examina o contexto completo de uma função ou procedimento. Através de análises detalhadas de fluxo de dados, o compilador consegue

identificar oportunidades de melhoria que não seriam visíveis em uma inspeção puramente local.

- **Otimização Interprocedural:** Atua de forma macro, atravessando as fronteiras entre os diferentes módulos e funções do programa. Isso possibilita técnicas agressivas como a substituição em linha (*inlining*), onde o corpo de uma função é inserido diretamente no local da chamada, eliminando os custos computacionais associados ao desvio de execução e troca de contexto (COOPER; TORCZON, 2012).

De maneira geral, a maioria das rotinas de otimização ocorre em duas fases complementares: primeiro, uma análise estática para mapear o fluxo de controle e dependências; segundo, a transformação sintática que efetivamente reescreve o código. Entre as manobras mais frequentes, destacam-se a eliminação de código morto (instruções inacessíveis), o deslocamento de cálculos para fora de laços e a simplificação de redundâncias aritméticas (AHO; SETHI; ULLMAN, 1995).

2.1.8 Geração de Código Objeto e Mapeamento de Arquitetura

A fase de geração de código objeto (ou geração de código de máquina) constitui a etapa final do processo de compilação, englobando as atividades finais do *back-end*. O principal objetivo desta fase é mapear a representação intermediária otimizada para a linguagem nativa da máquina alvo, garantindo que o código gerado seja logicamente correto, faça uso efetivo dos recursos de *hardware* disponíveis e execute de forma altamente eficiente (AHO; SETHI; ULLMAN, 1995).

Um dos maiores desafios enfrentados na geração de código objeto advém da diversidade arquitetônica dos processadores. Cada família de processadores (como as arquiteturas x86, ARM ou SPARC) dispõe de um conjunto próprio de instruções de máquina, também conhecido como Arquitetura do Conjunto de Instruções (ISA - *Instruction Set Architecture*). Isso implica que as operações de baixo nível e suas respectivas codificações binárias diferem radicalmente de um computador para outro (COOPER; TORCZON, 2012).

Para transpor essa barreira e evitar a necessidade de se construir um compilador inteiramente novo para cada arquitetura de *hardware* existente, os compiladores modernos adotam o princípio da *retargetabilidade* (capacidade de redirecionamento). Essa estratégia é viabilizada pelo uso do código intermediário: o *front-end* atua de forma independente de

máquina para gerar a representação intermediária, permitindo que a adaptação para um novo computador exija apenas a reescrita ou a substituição do módulo gerador de código (*back-end*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

O processo de geração de código específico para uma máquina baseia-se fortemente em duas atividades interligadas: a seleção de instruções e a alocação de registradores. A seleção de instruções atua como um reconhecedor de padrões que analisa a árvore ou a sequência linear do código intermediário e seleciona a sequência exata de operações na ISA do processador capaz de implementar aquele comportamento com o menor custo de execução possível. Em paralelo, a alocação de registradores gerencia o escasso conjunto de registradores físicos da Unidade Central de Processamento (UCP), decidindo quais valores em tempo de execução serão mantidos nestes componentes de altíssima velocidade e quais precisarão ser transferidos para a memória principal (PRICE; TOSCANI, 2001).

2.1.9 Execução de Código de Máquina

Após o processo de síntese do compilador, o programa objeto resultante é composto por uma sequência densa e linear de operações elementares codificadas diretamente em formato binário. Diferentemente da representação hierárquica e abstrata manipulada nas fases iniciais da compilação, como a árvore de derivação, o código de máquina não possui estruturas de alto nível; ele é estritamente de baixo nível, composto por instruções rudimentares que ocupam poucos *bytes* e determinam ações diretas no *hardware*, como a movimentação de valores entre endereços de memória e registradores, ou a adição de inteiros (PRICE; TOSCANI, 2001).

Para que esse código binário seja efetivamente processado, o programa precisa ser carregado na memória principal, momento em que um *software* carregador (*loader*) e o sistema operacional preparam o ambiente de tempo de execução, dividindo a memória em segmentos específicos, como a área do código estruturado e as áreas de dados. A partir desse ponto, a execução do algoritmo é ditada pela Unidade Central de Processamento (UCP), que utiliza um registrador especial da arquitetura, frequentemente chamado de Contador de Programa (*Program Counter* - PC) ou Ponteiro de Instrução (*Instruction Pointer* - IP), para rastrear rigorosamente a próxima instrução a ser executada (COOPER; TORCZON, 2012).

A dinâmica de execução da máquina ocorre através de um ciclo mecânico e contínuo. O processador avança através da fita de instruções, decodificando o código de operação (*opcode*)

de cada uma para compreender a semântica da ação requerida e, em seguida, executando-a em ordem. Nesse ambiente de baixo nível, o fluxo de controle, que em linguagens fontes é caracterizado por laços de repetição e desvios condicionais elaborados, é resolvido de maneira muito mais simples. A alteração do fluxo é realizada puramente por meio de operações de saltos (*jumps* ou *branches*), que escrevem um novo endereço de memória diretamente no Contador de Programa, desviando a execução de um ponto do código de máquina para outro (NYSTROM, 2021).

2.2 Engenharia de Software

Esta seção reúne as práticas de engenharia de software que orientaram a construção do sistema: a organização do trabalho em ciclos iterativos, a estratégia de testes automatizados, a separação de responsabilidades em camadas, as diretrizes de qualidade de código e a arquitetura do *framework* adotado no *back-end*.

2.2.1 Desenvolvimento Iterativo e Incremental e *Extreme Programming*

O gerenciamento clássico de projetos de software, frequentemente baseado no modelo em cascata (*waterfall*), assume que o desenvolvimento pode ser tratado como uma sequência previsível e linear de atividades. No entanto, a engenharia de software lida com sistemas intrinsecamente complexos e com alto grau de incerteza, e o engessamento de requisitos em fases iniciais costuma resultar em desperdício e na entrega de produtos que não atendem às necessidades reais. Em contraponto a essa visão, o desenvolvimento iterativo e incremental assume a imprevisibilidade natural do processo e organiza o trabalho em ciclos curtos, nos quais o sistema cresce por acréscimos funcionais sucessivos, cada um integrado e verificado antes do início do seguinte (PRESSMAN; MAXIM, 2016; SOMMERVILLE, 2018).

Entre os métodos ágeis, o *Extreme Programming* (XP) distingue-se por concentrar suas prescrições nas práticas técnicas de construção do software, e não em papéis ou cerimônias de gestão. O método parte do princípio de que o custo da mudança pode ser mantido baixo ao longo de todo o projeto, desde que o código permaneça simples, coberto por testes e continuamente refatorado. A mudança deixa, assim, de ser tratada como desvio a ser evitado e passa a ser incorporada ao fluxo normal de trabalho (ADAPTWORKS, 2012; BECK; ANDRES, 2004; WILDT et al., 2015).

As práticas que sustentam esse princípio incluem o planejamento incremental, no qual o escopo é detalhado apenas no momento em que será implementado; o *design* simples, que evita antecipar estruturas ainda não exigidas; o desenvolvimento guiado por testes, que estabelece o comportamento esperado antes da implementação; a refatoração contínua, que preserva a legibilidade do código à medida que ele cresce; a integração contínua, que reduz o custo de reunir contribuições paralelas; e a propriedade coletiva do código, segundo a qual qualquer integrante pode alterar qualquer parte do sistema, sustentada pela revisão mútua entre os desenvolvedores (BECK; ANDRES, 2004; WILDT et al., 2015).

Essa combinação torna o XP particularmente adequado a equipes reduzidas, nas quais não há massa crítica para instanciar papéis especializados: os mesmos desenvolvedores acumulam as atribuições de levantamento de requisitos, projeto, implementação, teste e revisão. Nesse cenário, as práticas técnicas assumidas pelo método ocupam o lugar dos mecanismos de coordenação que métodos orientados a processo delegariam a papéis distintos (WILDT et al., 2015). As subseções seguintes detalham as práticas que estruturaram diretamente a construção do núcleo de tradução: os testes automatizados e o *Test-Driven Development*, a separação de responsabilidades em camadas e as diretrizes de *Clean Code*.

2.2.2 Testes Automatizados e *Test-Driven Development* (TDD)

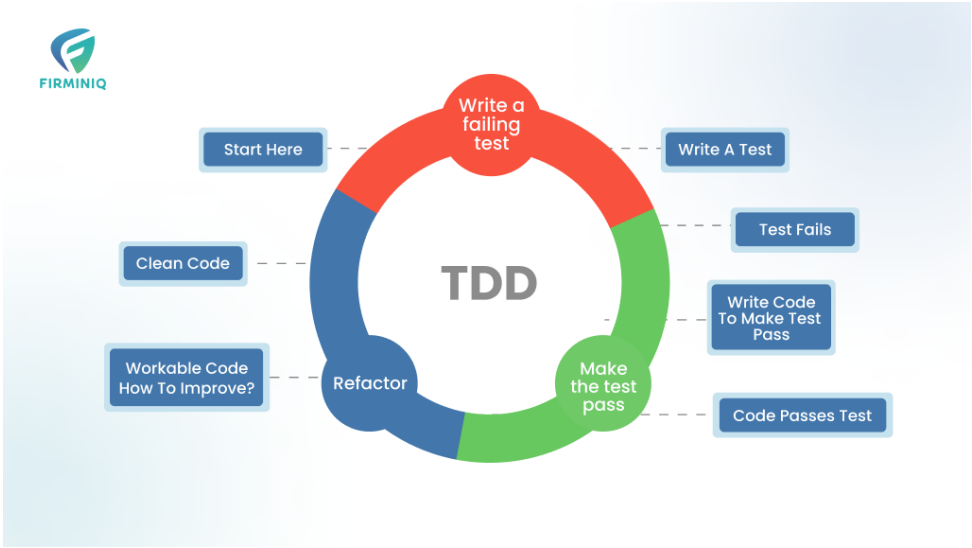
A garantia da qualidade de um software exige a validação contínua de seus comportamentos. Em projetos modernos, a automação de testes complementa a verificação manual e auxilia os desenvolvedores a detectar regressões nos comportamentos cobertos ao alterar, refatorar e evoluir o código. Essa prevenção contra a quebra de comportamentos já consolidados é conhecida como teste de regressão. Além de atestar o funcionamento da aplicação, os testes automatizados atuam como uma documentação viva e executável das regras de negócio do sistema (ANICHE, 2012).

Nesse contexto, destaca-se a prática do *Test-Driven Development* (Desenvolvimento Guiado por Testes). O TDD inverte o ciclo tradicional de engenharia, exigindo que o desenvolvedor escreva um teste automatizado antes mesmo de implementar a funcionalidade do sistema (ANICHE, 2012; MARTIN, 2011).

O processo segue um ciclo de três etapas fundamentais, muitas vezes chamado de ciclo vermelho-verde-refatora (*Red-Green-Refactor*), conforme ilustrado na Figura 2.3. Primeiro,

escreve-se um teste que falha, definindo o comportamento desejado; em seguida, escreve-se o código mais simples possível para fazer o teste passar; por fim, o código de produção é refatorado para remover duplicações e melhorar sua estrutura.

Figura 2.3 – Ciclo de Desenvolvimento Guiado por Testes (TDD).



Fonte: Firminiq (2024).

Mais do que uma simples técnica de verificação, o TDD é uma poderosa ferramenta de *design* de software. Ao escrever o teste primeiro, o desenvolvedor atua como o primeiro cliente do próprio código, o que o obriga a pensar na interface pública da classe, em suas dependências e em sua responsabilidade. Dificuldades na escrita de um teste, como a necessidade de configurações complexas ou o uso excessivo de dependências simuladas (*mock objects*), são indícios claros de problemas arquiteturais, denunciando alto acoplamento e baixa coesão estrutural (MARTIN, 2019; ANICHE, 2012).

Para maximizar a eficiência de execução e manutenção, a estratégia de testes de um sistema deve ser distribuída em diferentes níveis de granularidade, conceito frequentemente ilustrado pela Pirâmide de Testes. A base da pirâmide é composta pelos testes de unidade, que são extremamente rápidos, isolados e verificam o comportamento de pequenas partes do código, como classes ou métodos específicos. O nível intermediário abriga os testes de integração, responsáveis por garantir a correta comunicação entre os componentes internos e sistemas de infraestrutura externos, como bancos de dados ou serviços de terceiros. No topo da pirâmide estão os testes de ponta a ponta (*end-to-end*) e de aceitação, que validam o sistema inteiro através da interface com o usuário, simulando o comportamento real da aplicação de forma mais custosa e lenta (ANICHE, 2012; SILVEIRA et al., 2012).

Por fim, do ponto de vista arquitetural, os testes devem ser tratados como componentes essenciais do sistema. Contudo, para que não se tornem frágeis e difíceis de manter, eles devem ser desacoplados de elementos voláteis, como a Interface de Usuário (UI) ou as ferramentas de persistência. Uma arquitetura limpa e bem projetada permite que as regras de negócio centrais sejam exaustivamente testadas de forma isolada, garantindo execuções rápidas e *feedback* imediato para a equipe de desenvolvimento (MARTIN, 2019).

2.2.3 Arquitetura Limpa e Separação de Responsabilidades

O projeto arquitetural do sistema foi concebido sob a ótica da Separação de Responsabilidades (*Separation of Concerns* - SoC), dividindo o software em camadas isoladas com papéis independentes e bem definidos. A interface de usuário no *front-end* foca exclusivamente na experiência de uso e integração visual, enquanto a lógica de execução do negócio permanece no *back-end*. Isso respeita a premissa fundamental da Arquitetura Limpa de que as regras centrais da aplicação jamais devem depender de detalhes periféricos de entrega, considerando que a tecnologia *Web* ou a interface de usuário são apenas detalhes de implementação (MARTIN, 2019).

No que tange à camada de dados, a persistência foi totalmente abstraída por meio de ferramentas de Mapeamento Objeto-Relacional (ORMs, como *Prisma* e *SQLAlchemy*). O uso de ORMs atua como uma ponte que suaviza a discrepância entre o paradigma relacional e o orientado a objetos, mitigando a dependência direta de consultas SQL. Essa prática garante que o banco de dados seja tratado estritamente como um detalhe de infraestrutura de baixo nível, impedindo que sua estrutura tabular polua a lógica central e as regras de negócio do domínio (MARTIN, 2019; SILVEIRA et al., 2012).

Por fim, a maturidade da arquitetura expressa-se também pela adoção de Padrões de Projeto (*Design Patterns*) na construção do compilador. O uso do padrão *Factory*, por exemplo, foi empregado para encapsular a lógica complexa de instanciação dos componentes de análise (*Lexer* e *Scanners*). Essa abordagem centraliza a decisão de criação de objetos, favorecendo a coesão e reduzindo drasticamente o acoplamento global do sistema (SILVEIRA et al., 2012).

2.2.4 Qualidade de Código e *Clean Code*

A construção de um software ágil e sustentável esbarra invariavelmente na qualidade de sua base de código. O código-fonte é a linguagem formal na qual os requisitos de um sistema

são expressos e detalhados de modo que a máquina possa executá-los. No entanto, sob a pressão de prazos, é comum que as equipes produzam códigos desorganizados, resultando em uma degradação estrutural contínua que, a longo prazo, faz a produtividade despencar, podendo até mesmo inviabilizar a evolução do projeto (MARTIN, 2009).

Para combater essa entropia, a adoção das práticas de Código Limpo torna-se essencial. A escrita de um código limpo exige disciplina, o uso consciente de padrões e uma sensibilidade aguçada para reconhecer a desorganização e aplicar refatorações. Um código limpo deve ser elegante, simples, direto e, sobretudo, legível para os seres humanos. O profissionalismo no desenvolvimento de software pressupõe que o código não deve apenas funcionar, mas deve ser fácil de entender, alterar e estender por outros desenvolvedores (MARTIN, 2009).

Dentre as diretrizes estruturais básicas e inegociáveis para a manutenção dessa clareza na base de código, destacam-se:

- **Nomes Significativos:** A nomenclatura de variáveis, funções, parâmetros e classes deve revelar claramente a sua intenção e responder ao porquê de sua existência e ao que o componente faz. Deve-se evitar o uso de siglas obscuras, codificações desnecessárias ou trocadilhos que exijam mapeamentos mentais, facilitando assim a leitura fluida e a busca no código.

- **Funções Pequenas e Focadas:** Uma função deve ser extremamente concisa e projetada para fazer estritamente uma única coisa, mantendo um único nível de abstração. Funções extensas que realizam múltiplas tarefas, agrupam várias seções lógicas ou que misturam tratamento de erros com a lógica de negócio violam a organização e dificultam o entendimento e a testabilidade.

- **Uso Consciente de Comentários:** O uso de comentários costuma compensar a incapacidade do programador de se expressar claramente através do próprio código. Eles não devem ser utilizados para encobrir ou justificar um código ruim ou confuso. A energia da equipe deve ser direcionada para tornar o código tão claro e autoexplicativo que elimine a necessidade de anotações redundantes.

- **Eliminação de Duplicidade e Refatoração contínua:** A repetição de código é o grande inimigo de um sistema bem desenvolvido, pois representa trabalho, risco e complexidade desnecessária. Deve-se melhorar o design do código de forma contínua, aplicando

refatorações que tornem o sistema mais enxuto e compreensível, sem alterar o seu comportamento externo (MARTIN, 2009).

2.2.5 FastAPI: Framework para APIs de Alto Desempenho

O FastAPI é um *framework* web moderno e de alto desempenho projetado para a construção de Interfaces de Programação de Aplicações (APIs) utilizando a linguagem Python. Sua arquitetura foi desenhada para ser simples, rápida e eficiente, tirando proveito das funcionalidades modernas da linguagem, como as anotações de tipo explícitas (*type hints*) e o suporte nativo a operações da programação assíncrona (FASTAPI, 2026).

Um dos pilares que garantem a alta performance do FastAPI é a sua fundação sobre a especificação ASGI (*Asynchronous Server Gateway Interface*) e o *framework* Starlette. Diferentemente de *frameworks* web tradicionais que utilizam o modelo síncrono (WSGI), a arquitetura assíncrona do FastAPI permite que o servidor gerencie milhares de conexões simultâneas de forma eficiente, processando operações de entrada e saída (I/O) sem bloquear a *thread* de execução. Em ambientes de produção, aliado a servidores de aplicação como o Uvicorn, o FastAPI é capaz de suportar picos de mais de 20.000 requisições por segundo, apresentando desempenho comparável a linguagens compiladas como Go e plataformas como o NodeJS (DASROOT, 2026).

Outra característica central do *framework* é a integração profunda com a biblioteca Pydantic, que assume a responsabilidade pela validação e serialização rigorosa dos dados. Ao invés de exigir a aprendizagem de uma nova micro-linguagem de validação, o desenvolvedor utiliza os tipos de dados padrão do próprio Python. A ferramenta converte automaticamente os dados de entrada (como o corpo de uma requisição em formato JSON) para os tipos corretos e gera mensagens de erro estruturadas e detalhadas caso a validação falhe, garantindo uma maior segurança e consistência no processamento (FASTAPI, 2026; DUNOSSAURO, 2026).

Além disso, a plataforma é construída estritamente em torno de padrões abertos, notadamente a especificação OpenAPI e o JSON Schema. Essa aderência aos padrões permite que o FastAPI gere de forma automática e dinâmica a documentação interativa da API, disponibilizando interfaces prontas para o consumo do desenvolvedor, como o Swagger UI e o ReDoc. Isso reduz drasticamente o tempo gasto com a documentação manual, facilita a integração com sistemas de *front-end* e permite a geração automática de clientes (*SDKs*) em

diversas linguagens (FASTAPI, 2026).

Por fim, o *framework* incorpora um sistema de Injeção de Dependência poderoso, porém intuitivo, que simplifica a integração do código da aplicação com bancos de dados relacionais (através de ORMs como o SQLAlchemy), além de prover mecanismos nativos e facilitados para a implementação de segurança e autenticação, como fluxos OAuth2 e *tokens* JWT. Toda essa gama de funcionalidades faz do FastAPI uma escolha robusta, tanto para a criação rápida de protótipos quanto para sistemas corporativos altamente escaláveis (DUNOSSAURO, 2026).

2.2.6 Testes de integração de APIs assíncronas

Os testes de uma API podem verificar conjuntamente a interpretação da requisição, as regras de negócio, a persistência e a resposta. O pytest permite organizar a preparação e a liberação de recursos em *fixtures*, compartilhadas pelos cenários que delas dependem. Para funções e recursos assíncronos, o plugin pytest-asyncio oferece integração com o ciclo de eventos e um modo automático de identificação dos testes. Esses mecanismos permitem explicitar as condições iniciais e reduzir a repetição do código de preparação (pytest development team, 2026; pytest-asyncio contributors, 2026).

A documentação do FastAPI apresenta testes assíncronos com `AsyncClient` e `ASGITransport`, do HTTPX, para enviar requisições diretamente à aplicação. O transporte em processo dispensa um servidor HTTP externo nesse tipo de verificação. A substituição de dependências permite fornecer recursos próprios dos testes, como uma sessão de banco de dados, preservando as rotas e os serviços exercitados. Essa abordagem oferece um recorte de integração, cujo alcance depende dos componentes reais e substituídos no ambiente (FastAPI, 2026a; FastAPI, 2026b).

2.2.7 Contratos de resposta e minimização dos dados

Uma resposta de API deve conter os campos necessários à operação e permitidos ao solicitante. A OWASP relaciona a exposição indevida de propriedades e a alteração de atributos sem permissão ao risco de autorização em nível de propriedade. Entre as medidas recomendadas estão selecionar explicitamente os campos de saída, restringir os atributos atualizáveis e validar as respostas por esquema. Enviar um objeto completo e ocultar campos na interface mantém esses dados acessíveis ao cliente (OWASP Foundation, 2023b).

No FastAPI, o parâmetro `response_model` declara o contrato de saída e permite validar, serializar e filtrar o resultado de acordo com esse contrato. Modelos distintos para entrada, resumo e detalhe tornam explícito o que cada operação recebe ou devolve. A documentação exemplifica a separação entre os dados usados para cadastrar um usuário e aqueles retornados na resposta, excluindo a senha do modelo de saída (FastAPI, 2026a).

Neste trabalho, a redução dos dados retornados é tratada como seleção de campos e minimização da resposta. Essa decisão é distinta da validação dos valores recebidos e do tratamento de conteúdo para apresentação. Por exemplo, omitir uma propriedade não sanitiza o HTML eventualmente contido em outra. A OWASP distingue a validação sintática e semântica das medidas específicas de prevenção de injeções e da codificação de saída adequada ao contexto de uso (OWASP Foundation, 2026b).

2.2.8 Autorização e proteção de recursos no back-end

A autenticação identifica o solicitante; a autorização determina quais ações ele pode realizar. A OWASP recomenda o menor privilégio, a negação por padrão e a verificação de permissões em cada requisição. Quando uma operação recebe o identificador de um recurso, deve verificar também a relação entre esse recurso, a ação e o usuário autenticado. Assim, possuir uma sessão válida não basta para consultar ou modificar qualquer registro (OWASP Foundation, 2026a; OWASP Foundation, 2023a).

A proteção das credenciais e do transporte complementa esses controles. Senhas devem ser armazenadas mediante funções próprias de derivação de hash, e tokens assinados precisam ter sua integridade e validade verificadas. A assinatura de um JWT não equivale à criptografia de seu conteúdo; o transporte protegido por HTTPS continua necessário. Essas medidas protegem aspectos diferentes do fluxo de autenticação e não substituem as decisões de autorização dos serviços (OWASP Foundation, 2026c; OWASP Foundation, 2026d).

Outro aspecto é limitar o consumo de recursos. A OWASP recomenda impor limites ao tamanho de entradas, à quantidade de registros retornados e à frequência das operações. A paginação pode restringir o volume de uma resposta, mas sua eficácia depende de como a consulta é implementada e de sua aplicação aos caminhos disponíveis. Esses limites devem ser considerados juntamente com testes das permissões, incluindo tentativas de acesso que precisam ser negadas (OWASP Foundation, 2023c; OWASP Foundation, 2026a).

2.3 Softwares Educativos e Tecnologias de Apoio à Aprendizagem

Uma plataforma educativa é um ambiente virtual desenvolvido para apoiar processos de ensino e aprendizagem, oferecendo recursos, ferramentas e conteúdos que facilitam a interação entre alunos, professores e instituições de ensino. Nesse cenário de transformação, a escolha da ferramenta certa não é apenas uma questão tecnológica, mas um fator determinante para o engajamento, a motivação e o sucesso educacional (BENEDETTI, 2025).

Contudo, para aproveitar o potencial das novas tecnologias, a instituição de ensino precisa ir além da simples adoção de ferramentas digitais. É essencial entender o propósito pedagógico por trás de cada inovação, garantindo que a tecnologia esteja sempre a serviço da aprendizagem e não o contrário. Afinal, a tecnologia educacional atua como um meio que exige professores preparados para usar recursos digitais de forma criativa e estratégica, conectando os alunos a um aprendizado relevante, dinâmico e significativo (Mind Makers, 2025).

Quando projetada com esse foco pedagógico, uma plataforma educativa bem estruturada permite a personalização do aprendizado, ajustando-se ao ritmo e ao estilo de cada aluno, o que maximiza seu potencial e promove uma assimilação muito mais eficaz do conteúdo. Essa abordagem valoriza o tempo de cada estudante e amplia as suas capacidades ao longo de toda a jornada educacional. Além disso, a integração de dinâmicas interativas nas ferramentas de ensino, como a gamificação e a aprendizagem baseada em desafios, transforma a educação em um processo envolvente de recompensas. Esses sistemas lúdicos ajudam a despertar o interesse contínuo dos alunos e a desenvolver habilidades socioemocionais indispensáveis, como persistência e colaboração (BENEDETTI, 2025; Mind Makers, 2025).

2.4 Trabalhos Relacionados

Esta seção compara ferramentas de iniciação em programação e projetos de construção de compiladores com o núcleo desenvolvido neste trabalho. A análise considera três eixos: escolhas de interface, recursos pedagógicos e estratégias de interação com o estudante. A possibilidade de personalização da linguagem pelo usuário é examinada como característica adicional na comparação final.

2.4.1 Critérios de Seleção dos Trabalhos

A seleção é intencional e delimitada aos dois eixos do projeto: o uso de linguagens para aprender programação e a construção de seus mecanismos de tradução. Não constitui uma revisão sistemática nem um levantamento exaustivo das ferramentas disponíveis. Foram adotados três critérios de inclusão: (i) finalidade educacional explícita no ensino de programação ou de compiladores; (ii) disponibilidade de documentação, código-fonte ou publicação que permita examinar o funcionamento; e (iii) relação com edição e execução de programas, apresentação da sintaxe ou inspeção das etapas de tradução.

A aplicação desses critérios ao conjunto selecionado é explicitada a seguir:

- **Portugol Studio e Portugol Webstudio:** representam a iniciação por linguagem textual em português e permitem comparar edição, execução e recursos de inspeção de programas. O Studio fornece depuração e inspeção de variáveis; o Webstudio integra a discussão como sua alternativa de uso pelo navegador (UNIVALI, 2026; MARTINS, 2025).

- **Quorum:** permite discutir escolhas de sintaxe investigadas experimentalmente e recursos de acessibilidade, aproximando a comparação da hipótese sobre dificuldades sintáticas de iniciantes (STEFIK; SIEBERT, 2013; QUORUM, 2025).

- **Beecrowd:** representa a prática por exercícios com avaliação automatizada de entrada e saída. Sua inclusão decorre da relação com o fluxo de submissão de exercícios da plataforma, e não de oferecer personalização da sintaxe (FERREIRA; SILVA; RIBEIRO, 2025; beecrowd, 2026).

- **Laila:** representa a construção de analisadores léxicos e sintáticos em ambiente web, permitindo comparar a exposição do processo de tradução e a forma de interação com as especificações da linguagem (SILVA; PEREIRA; LIMA, 2021).

- ***chibicc*:** fornece código-fonte e histórico incremental de um compilador C, permitindo comparar a organização das etapas de análise e geração de código com a implementação do núcleo deste trabalho (UEYAMA, 2020).

O Scratch, citado na introdução como exemplo de iniciação por blocos, não integra a comparação detalhada porque o recorte escolhido prioriza linguagens textuais, submissão de código e construção de tradutores. Essa delimitação não implica ausência de valor pedagógico

ou de extensibilidade no Scratch (Scratch Foundation, 2026). Também ficam fora do recorte IDEs de uso geral sem uma proposta educacional específica e ferramentas para as quais não se dispõe de material suficiente para examinar os três eixos. Não se apresenta uma contagem de exclusões, pois não houve uma busca sistemática com conjunto inicial de candidatos registrado.

2.4.2 Ambientes Voltados à Iniciação em Programação

Reúnem-se aqui as ferramentas cujo público-alvo é o estudante em primeiro contato com a programação e cujo propósito declarado é reduzir o atrito desse contato inicial. Cada uma é examinada segundo os três eixos anunciados no início desta seção: as escolhas de interface, os recursos pedagógicos e as estratégias de interação com o estudante.

Portugol Studio e Portugol Webstudio

Escolhas de interface. O Portugol Studio é um Ambiente de Desenvolvimento Integrado (IDE) educacional de código aberto, projetado e mantido pelo Laboratório de Inovação Tecnológica na Educação (LITE) da UNIVALI, cuja composição de tela abstrai a complexidade dos ambientes profissionais sem abandonar sua organização geral. Visando reduzir barreiras tecnológicas, o projeto evoluiu para o Portugol Webstudio, versão que permite compilar e executar códigos diretamente no navegador, dispensando qualquer instalação no computador do usuário (UNIVALI, 2026; MARTINS, 2025).

Recursos pedagógicos. O recurso central é a linguagem Portugol, com sintaxe baseada em C e PHP, porém com comandos totalmente estruturados na língua portuguesa, o que elimina a exigência de operar em dois idiomas simultaneamente. A esse núcleo somam-se recursos didáticos de visualização: execução passo a passo por meio de depurador, inspetor de variáveis em tempo real e árvore estrutural do código, que tornam observável o comportamento interno da máquina (UNIVALI, 2026; GOMES, 2020).

Estratégias de interação. A interação apoia-se no acompanhamento visual da execução, permitindo ao estudante confrontar o algoritmo imaginado com o efetivamente executado. O suporte nativo a bibliotecas gráficas e de som amplia essa estratégia ao viabilizar a criação de jogos com interface gráfica, cuja saída fornece retorno imediato e concreto sobre o efeito do código escrito (GOMES, 2020; MARTINS, 2025).

Quorum: linguagem de programação baseada em evidências

Escolhas de interface. A equipe do projeto desenvolveu o Quorum Studio, um Ambiente

de Desenvolvimento Integrado nativamente acessível, isto é, concebido desde o início para operar com leitores de tela, e não adaptado a eles posteriormente. A infraestrutura é flexível: a linguagem executa tanto *offline*, gerando *bytecode* para a Máquina Virtual Java, quanto diretamente no navegador do usuário (LADNER, 2019; QUORUM, 2025).

Recursos pedagógicos. A linguagem Quorum foi inicialmente concebida pelo pesquisador Andreas Stefik com o objetivo de ser nativamente acessível a estudantes cegos ou com deficiência visual, e evoluiu para uma linguagem de propósito geral mantendo sua premissa principal: ser orientada a evidências. Suas regras sintáticas e semânticas não decorrem de tradição ou intuição, mas de ensaios controlados com usuários novatos e experientes. Os experimentos compararam o desempenho de novatos em linguagens comerciais com o obtido em uma linguagem de controle cujas palavras-chave foram geradas aleatoriamente, isolando o efeito da forma sintática. Os resultados indicaram que remover símbolos obscuros — chaves, colchetes e pontos e vírgulas — e substituí-los por palavras de uso comum reduz consistentemente as taxas de erro sintático, o que se materializa em construções como `repeat 10 times` (STEFIK; SIEBERT, 2013).

Estratégias de interação. A modalidade primária de interação prevista é auditiva: a linguagem foi projetada para que sua leitura por síntese de voz permaneça clara, exigência que condiciona desde a escolha das palavras-chave até a pontuação. O ecossistema estende essa interação a domínios de saída variados, suportando desde lógicas simples até jogos, processamento de áudio e robótica, o que oferece formas de perceber o resultado do programa que não dependem exclusivamente da leitura visual (LADNER, 2019; QUORUM, 2025).

Beecrowd: prática de algoritmos e avaliação automatizada

Escolhas de interface. O Beecrowd, anteriormente conhecido como URI Online Judge, é uma plataforma *online* organizada em torno de um acervo navegável por categorias e níveis de dificuldade. A interface não oferece ambiente de desenvolvimento: o estudante redige a solução em ferramenta externa e submete o código pronto, o que pressupõe um ambiente já configurado (GARCIA, 2024).

Recursos pedagógicos. O acervo conta com milhares de desafios algorítmicos, das lógicas fundamentais às estruturas de dados complexas, e admite submissões em diversas linguagens de mercado, como C++, Python, Java e JavaScript. A graduação de dificuldade é o principal recurso didático, pois permite progressão por incrementos e seleção, pelo professor, de conjuntos

coerentes com o momento da disciplina (GARCIA, 2024; FERREIRA; SILVA; RIBEIRO, 2025).

Estratégias de interação. A característica definidora é o sistema de juiz *online*: o aluno lê a especificação do problema, desenvolve o código e o submete; em segundos, o sistema compila, executa e testa a solução contra casos de teste fechados, devolvendo vereditos como "*Accepted*", "*Wrong Answer*" ou "*Presentation Error*". Cabe observar que esse retorno informa o resultado, mas não localiza o erro no código. Aliada a elementos de gamificação como fóruns e *rankings*, a validação em tempo real estimula a prática contínua e desloca o aluno de uma posição passiva para a resolução ativa de problemas (FERREIRA; SILVA; RIBEIRO, 2025; MÃO NO CÓDIGO, 2022).

2.4.3 Projetos Didáticos de Construção de Compiladores

Reúnem-se aqui as iniciativas voltadas ao ensino da construção de tradutores. Embora se dirijam a um público mais avançado, suas decisões técnicas e pedagógicas dialogam diretamente com o núcleo de tradução que sustenta a plataforma proposta, e por isso são examinadas segundo os mesmos três eixos.

Laila: gerador de analisadores léxicos e sintáticos *online*

Escolhas de interface. A ferramenta oferece um ambiente de desenvolvimento integrado totalmente *online*. O módulo de interação com o usuário foi construído em JavaScript com o *framework* VueJS e a biblioteca CodeMirror, garantindo edição de código com destaque de sintaxe e identificação de erros em tempo real; o processamento ocorre por meio de uma API REST em Python, que executa FLEX e Bison no servidor, captura falhas como laços infinitos ou erros de compilação e devolve as saídas ao navegador. A escolha responde a um obstáculo concreto: o ensino prático da disciplina exige a instalação e a configuração de ferramentas como GCC, FLEX e Bison, tarefas árduas em sistemas operacionais não baseados em Linux (SILVA; PEREIRA; LIMA, 2021).

Recursos pedagógicos. O recurso didático central é permitir que o aluno projete, escreva e teste seus analisadores léxicos e sintáticos sem qualquer configuração local, concentrando o tempo de laboratório no objeto de estudo e não em sua preparação (SILVA; PEREIRA; LIMA, 2021).

Estratégias de interação. A interação organiza-se em um ciclo curto entre edição, submissão e retorno do resultado. Em estudos de caso realizados com turmas de Ciência da

Computação, os participantes relataram melhoria na compreensão dos algoritmos de análise léxica e sintática, atribuída à praticidade de testar as linguagens em tempo real, e o uso da plataforma contornou os problemas de compatibilidade de ambiente (SILVA; PEREIRA; LIMA, 2021).

***chibicc*: abordagem incremental para o ensino de compiladores**

Escolhas de interface. Este é o caso em que a análise por eixos revela a maior distância em relação às demais ferramentas: o *chibicc* não possui interface própria. Trata-se de um repositório de código-fonte, cuja fruição pressupõe editor, cadeia de compilação e sistema de controle de versão já instalados e dominados pelo estudante. Toda a mediação recai sobre o histórico de *commits* do projeto (UEYAMA, 2020).

Recursos pedagógicos. O diferencial pedagógico reside na abordagem incremental. Em vez de apresentar uma arquitetura monolítica desde o início, o projeto orienta a construção do compilador passo a passo: o aluno começa por um tradutor rudimentar, capaz de aceitar apenas um único número como linguagem válida, e a partir dessa base mínima acrescenta, de forma isolada a cada etapa, novos recursos gramaticais e semânticos, até que a linguagem aceita corresponda à especificação do C11. Apesar do enfoque educacional, o resultado suporta ampla variedade de recursos dessa especificação, sendo capaz de compilar sem modificações programas de grande porte, como o sistema de controle de versão Git e o banco de dados SQLite (UEYAMA, 2020).

Estratégias de interação. Não há retorno automatizado nem execução assistida: a interação consiste na leitura do código de cada etapa e em sua reimplementação pelo estudante, que verifica o próprio progresso executando manualmente a suíte de testes do projeto. O método desmistifica a transição do código-fonte para o código de máquina, mas transfere integralmente ao aprendiz a responsabilidade pela regulação do próprio ritmo (UEYAMA, 2020).

2.4.4 Análise Crítica e o Diferencial da Solução Proposta

A Tabela 2.1 sintetiza as ferramentas examinadas segundo os três eixos adotados nesta seção, acrescentando a coluna que expressa o traço distintivo da proposta: a possibilidade de o próprio estudante redefinir a sintaxe da linguagem.

Tabela 2.1 – Comparativo entre as ferramentas analisadas e a plataforma proposta, segundo os três eixos de análise.

Ferramenta	**Escolhas de interface**	**Recursos pedagógicos**	**Estratégias de interação**	**Personalização da sintaxe pelo estudante**
Portugol Studio e Webstudio	IDE própria, em versão local e em versão executada no navegador	Linguagem com palavras-chave em português; depurador, inspetor de variáveis e árvore sintática	Acompanhamento visual da execução; projetos gráficos e sonoros	Não
Quorum	IDE própria concebida para leitores de tela; execução local ou no navegador	Sintaxe definida por evidência empírica; operadores expressos por palavras	Leitura por síntese de voz; saídas em áudio, gráficos e robótica	Não
Beecrowd	Plataforma web sem ambiente de edição; exige ferramenta externa	Acervo graduado por tema e dificuldade; múltiplas linguagens de mercado	Ciclo submissão–veredito, sem localização do erro; gamificação	Não
Laila	IDE web com destaque de sintaxe e diagnóstico *inline*	Dispensa a instalação de FLEX, Bison e GCC, liberando o tempo de laboratório	Ciclo curto entre edição, execução e diagnóstico	Sim, mas como objeto de estudo: o aluno escreve gramáticas formais
chibicc	Sem interface própria; repositório de código-fonte	Trajetória incremental, uma capacidade da linguagem por etapa	Leitura e reimplementação do código; sem retorno automatizado	Não
Plataforma proposta	IDE web própria, com terminal integrado e sistema de arquivos	Exposição das fases de tradução (*tokens* e código intermediário); avaliação automatizada de exercícios	Execução e depuração passo a passo; erros relatados no vocabulário escolhido pelo próprio estudante	Sim: palavras-chave, delimitadores de bloco, operadores, literais booleanos e terminador de instrução

Fonte: elaborado pelo autor.

A comparação evidencia propósitos distintos. Portugol Studio e Quorum oferecem

linguagens voltadas à aprendizagem, enquanto Beecrowd organiza a prática por exercícios e vereditos automatizados. Esses recursos respondem a necessidades diferentes: compreender a execução de um programa, escolher uma sintaxe acessível e verificar uma solução contra casos de teste. Sua existência impede tratar a tradução para o português ou a simples execução de exercícios como diferenciais exclusivos da proposta (UNIVALI, 2026; QUORUM, 2025; beecrowd, 2026).

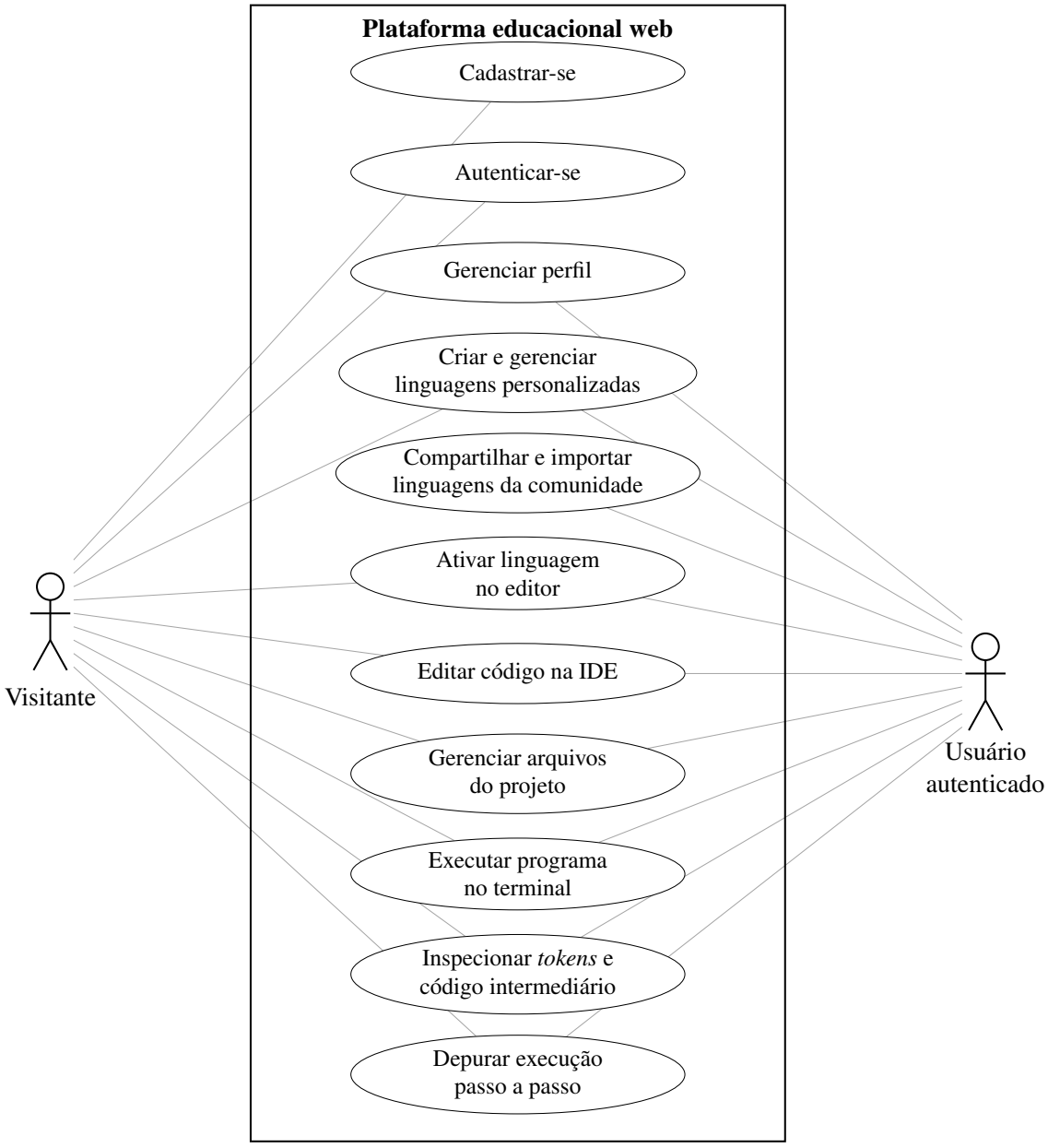
Laila e *chibicc* aproximam-se do eixo de construção de tradutores. A primeira permite elaborar analisadores com FLEX e Bison pelo navegador; o segundo expõe a implementação incremental de um compilador C. O núcleo deste trabalho combina uma representação intermediária observável com um interpretador e com parâmetros de configuração já implementados, sem exigir que o estudante escreva um novo analisador para cada escolha de vocabulário (SILVA; PEREIRA; LIMA, 2021; UEYAMA, 2020).

O diferencial técnico proposto é reunir, na mesma IDE, a configuração desses elementos da linguagem, a inspeção dos *tokens* e das instruções intermediárias e a execução do programa. A comparação de funcionalidades não demonstra superioridade pedagógica. Em particular, não permite concluir que a plataforma reduza a carga cognitiva, a reprovação ou o tempo de aprendizagem. Tais efeitos permanecem como hipóteses que exigem avaliação empírica, conforme a delimitação apresentada na introdução.

3 PROPOSTA

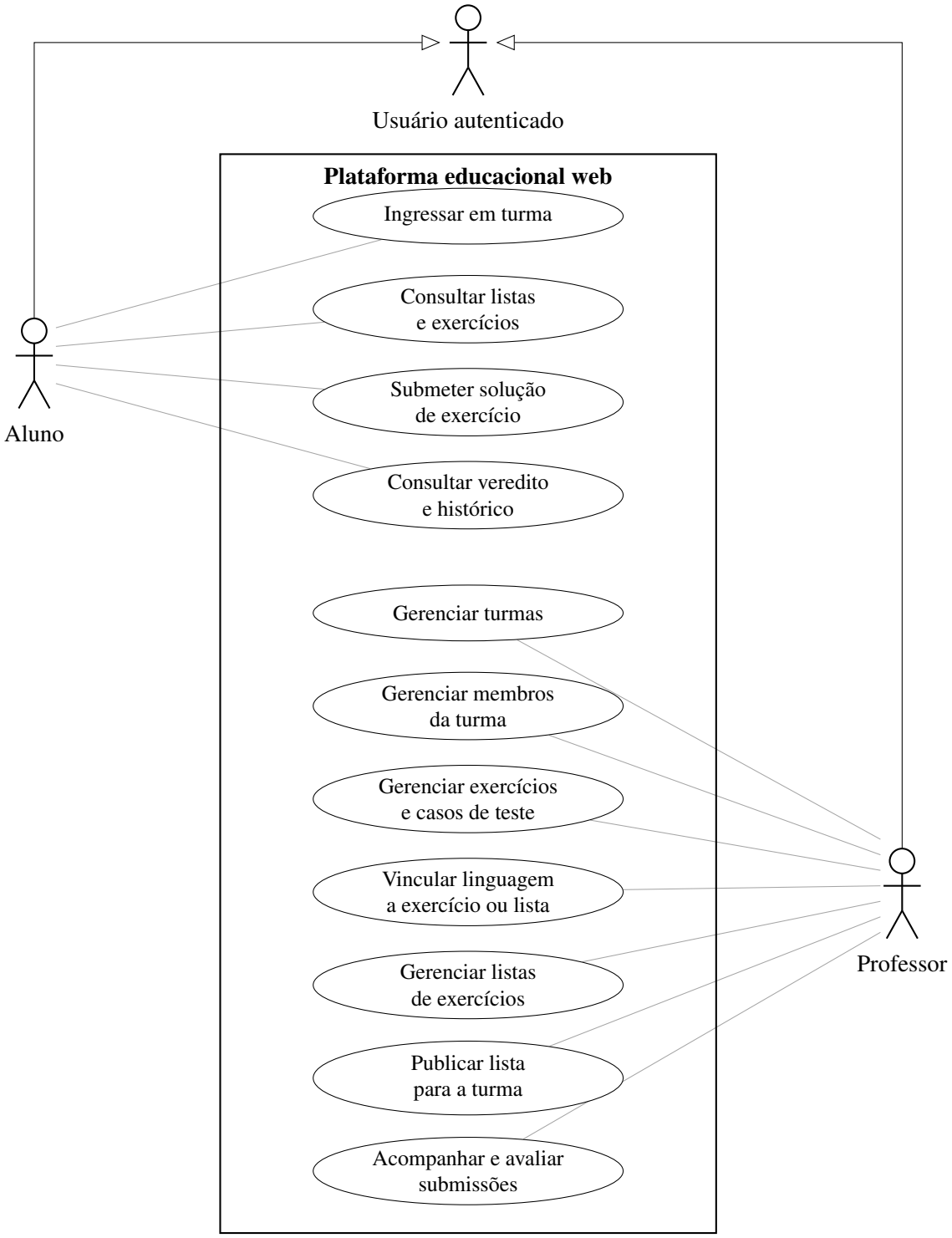
Este capítulo apresenta o escopo do trabalho e o cronograma de desenvolvimento das duas etapas do TCC. O projeto consiste em construir uma plataforma educacional web composta por uma IDE acoplada a um interpretador dinâmico. A plataforma permite ao usuário personalizar o vocabulário da linguagem, os delimitadores de bloco, os operadores em forma de palavra, os literais booleanos e as regras de finalização de instrução. O objetivo é reduzir a dificuldade inicial de aprender programação deslocando o foco do aluno da sintaxe para a lógica. O conjunto completo de funcionalidades previstas para a plataforma, com os respectivos atores, é apresentado nas Figuras 3.1 e 3.2. A Figura 3.1 situa o núcleo de tradução tratado neste texto dentro do escopo funcional mais amplo do projeto: as funcionalidades de edição, execução e inspeção do código, nessa figura, são as que acionam diretamente o interpretador descrito nos capítulos seguintes.

Figura 3.1 – Casos de uso: acesso e ambiente de desenvolvimento.



Fonte: elaborado pelo autor.

Figura 3.2 – Casos de uso: participação e gestão pedagógica.



Fonte: elaborado pelo autor.

A partir desses casos de uso, consolidaram-se os requisitos que orientaram a construção da plataforma. A Tabela 3.1 relaciona os requisitos funcionais, isto é, o que o sistema deve fazer; a Tabela 3.2, os requisitos não funcionais, que estabelecem as qualidades esperadas da solução e os critérios sob os quais ela será verificada.

Tabela 3.1 – Requisitos funcionais da plataforma.

ID	Descrição
RF01	Permitir o cadastro e a autenticação de usuários, distinguindo os papéis de aluno e de professor
RF02	Permitir ao usuário autenticado consultar e editar os dados de seu perfil
RF03	Oferecer um assistente guiado, organizado em etapas independentes, para a criação de uma linguagem personalizada
RF04	Permitir a redefinição dos elementos da linguagem relacionados na Tabela 4.1
RF05	Validar preventivamente cada configuração, impedindo colisões entre palavras-chave, operadores, literais, delimitadores e terminador de instrução
RF06	Exibir, em tempo real, a pré-visualização de um programa exemplo sob a configuração corrente
RF07	Persistir localmente a customização ativa e remotamente as linguagens nomeadas pelo usuário
RF08	Permitir publicar uma linguagem no catálogo comunitário e importar linguagens publicadas por terceiros
RF09	Oferecer edição de código com realce de sintaxe e sugestão automática ajustados à linguagem ativa
RF10	Manter um sistema de arquivos virtual que permita criar, renomear, remover e alternar entre arquivos
RF11	Executar o programa produzido no editor, com entrada e saída conectadas ao terminal integrado
RF12	Exibir ao usuário a tabela de *tokens* reconhecidos e o código intermediário gerado
RF13	Oferecer execução passo a passo, destacando no editor a instrução em processamento
RF14	Relatar erros léxicos, sintáticos, semânticos e de execução no vocabulário da linguagem ativa e no idioma selecionado
RF15	Permitir ao professor criar e gerir turmas, e ao aluno ingressar em uma turma
RF16	Permitir ao professor criar e gerir exercícios, com casos de teste de entrada e saída esperada
RF17	Permitir vincular uma configuração de linguagem a um exercício ou lista, restringindo o vocabulário disponível ao aluno
RF18	Permitir organizar exercícios em listas e publicá-las para uma turma
RF19	Avaliar automaticamente a submissão do aluno contra os casos de teste do exercício
RF20	Permitir ao professor acompanhar as submissões da turma e atribuir nota

Fonte: elaborado pelo autor.

O desenvolvimento foi dividido em seis etapas principais, parcialmente sobrepostas: (I) revisão bibliográfica sobre interpretadores, compiladores e ferramentas educacionais; (II) definição dos requisitos e da arquitetura do sistema; (III) implementação da análise léxica e sintática com suporte ao vocabulário configurável e às variantes predefinidas de escrita; (IV) geração e execução do código intermediário e verificação semântica; (V) integração entre o núcleo

Tabela 3.2 – Requisitos não funcionais da plataforma.

ID	Descrição
RNF01	**Acessibilidade:** os componentes de interface devem observar as diretrizes WAI-ARIA (W3C, 2023), com navegação por teclado, leitura por tecnologias assistivas e gerenciamento de foco
RNF02	**Responsividade:** as composições de tela devem reorganizar-se conforme a faixa de resolução do dispositivo
RNF03	**Internacionalização:** toda cadeia visível ao usuário deve estar externalizada, com suporte aos idiomas *pt-BR*, *pt-PT*, *es* e *en*
RNF04	**Autonomia de execução:** edição, análise léxica e sintática, geração de código intermediário e interpretação devem permanecer funcionais na ausência de conexão com o serviço de persistência
RNF05	**Latência:** a execução do núcleo de tradução deve ocorrer sem ciclo de requisição e resposta a servidor remoto
RNF06	**Consistência visual:** os temas claro e escuro devem manter paridade de contraste e legibilidade, inclusive no editor
RNF07	**Testabilidade:** as construções críticas devem ser cobertas por testes automatizados, executados localmente e no *pipeline* de integração contínua
RNF08	**Segregação de acesso:** as rotas e operações restritas devem verificar o papel do usuário antes de permitir o acesso

Fonte: elaborado pelo autor.

do interpretador e a plataforma web; e (VI) testes automatizados do núcleo de tradução e definição do protocolo de avaliação junto a usuários, cuja aplicação é registrada como encaminhamento futuro. O cronograma dessas etapas e das atividades de redação está na Tabela 3.3, onde cada coluna numerada representa um mês corrido e as células marcadas com "X" indicam os meses de execução previstos.

Este trabalho foi desenvolvido em dupla, conforme a pré-proposta aprovada, que distribui o projeto em **funções complementares**: o desenvolvimento da plataforma web e da interface de personalização da linguagem ficou a cargo de Igor Lamoia Queiroz, enquanto a concepção e a implementação do **núcleo do interpretador** (análise léxica, sintática e semântica, geração e execução de código intermediário) constituem a demanda desta monografia. As etapas iniciais de revisão bibliográfica e de definição da arquitetura geral foram conduzidas conjuntamente, e o regime de dupla manifestou-se sobretudo nas atividades de fronteira entre os subsistemas: a definição dos contratos de comunicação entre os módulos e a integração do núcleo à plataforma, em que decisões de uma camada exigiam ajustes na outra. O aprofundamento técnico e analítico do núcleo de tradução, objeto de avaliação deste documento, é de responsabilidade individual do autor.

A divisão de eixos de aprofundamento individual, prevista na pré-proposta, orienta o conteúdo técnico do Capítulo 4, especialmente as Seções 4.3 e 4.4, e é detalhada no Apêndice A.

Tabela 3.3 – Cronograma de desenvolvimento do trabalho de conclusão de curso.

Atividade	**TCC I**						**TCC II**					
	01	**02**	**03**	**04**	**05**	**06**	**07**	**08**	**09**	**10**	**11**	**12**
Revisão Bibliográfica	X	X	X	X								
Escrita da Introdução e Revisão de Literatura		X	X	X								
Definição de requisitos e dos elementos configuráveis da linguagem			X	X								
Escrita da Metodologia			X		X	X	X					
Arquitetura e Modelagem do Interpretador			X	X								
Implementação da Análise Léxica e Sintática				X	X							
Geração e Execução de Código Intermediário e Análise Semântica				X	X	X						
Integração entre a plataforma e interpretador					X	X	X					
Testes Automatizados e Definição do Protocolo de Avaliação									X	X	X	
Escrita do TCC (Resultados, Discussão e Conclusão)									X	X	X	X

Fonte: elaborado pelo autor.

Neste documento, o foco está no núcleo de tradução: as fases de análise léxica, sintática e semântica, a geração de código intermediário e o funcionamento do interpretador. O aluno Igor Lamoia Queiroz aprofunda, em seu documento, a IDE web: a interface de personalização da linguagem, a experiência de uso da plataforma e o protocolo de coleta de *feedback* dos estudantes. Essa divisão se justifica pela amplitude do projeto, que seria difícil de cobrir com profundidade suficiente por apenas um autor.

Com o TCC I concluído, o TCC II dará continuidade ao aprofundamento do núcleo de tradução, ampliando a cobertura de testes e a avaliação técnica do interpretador (a correção do código intermediário gerado e a robustez frente às variações configuráveis da linguagem) e consolidando a análise comparativa entre a solução desenvolvida e as ferramentas discutidas no referencial teórico, sob a ótica das capacidades do núcleo de tradução e execução. Essa comparação evidencia a convergência da proposta com sistemas relacionados de compiladores, pois o projeto também explicita as etapas de análise léxica, análise sintática e geração de representação intermediária; contudo, diferencia-se ao integrar essas etapas a uma linguagem configurável e a uma IDE educacional voltada ao estudante iniciante. O próximo capítulo descreve a modelagem e a implementação realizadas até o fim do TCC I.

4 METODOLOGIA

Este capítulo descreve a abordagem metodológica adotada para o desenvolvimento do interpretador dinâmico proposto, detalhando a arquitetura geral do sistema, as decisões de projeto que sustentam cada fase do compilador implementado e os mecanismos de integração com a plataforma web. A condução do trabalho seguiu as etapas previstas no cronograma do projeto, contemplando, na revisão para a segunda fase do Trabalho de Conclusão de Curso, a revisão bibliográfica, a especificação do escopo da linguagem e de seus elementos configuráveis, a modelagem da arquitetura do interpretador, a implementação das análises léxica e sintática, a geração e a execução do código intermediário e a integração entre o núcleo do compilador e o ambiente web.

A descrição da implementação foi conferida inicialmente em 16 de setembro de 2026, na revisão `b4372bf`. Em 19 de setembro, a análise foi ampliada para a revisão `e0a012b`, contemplando os testes do serviço Python, os contratos de resposta, os controles de acesso e o fluxo de integração contínua. Os resultados locais da nova execução são identificados nas seções de testes; as capturas anteriores conservam seu caráter histórico (QUEIROZ; SILVA, 2026b; QUEIROZ; SILVA, 2026c; QUEIROZ; SILVA, 2026a; SILVA, 2026b).

A construção do sistema foi conduzida de forma incremental e iterativa, orientada pelas práticas técnicas do *Extreme Programming* discutidas na Seção 2.2.1. Convém explicitar de que modo essas práticas foram efetivamente instanciadas em uma equipe de dois integrantes, uma vez que o método foi adotado de forma parcial e deliberada. Não se estabeleceram ciclos de duração fixa: o trabalho partiu de um conjunto mínimo de requisitos e avançou por acréscimos funcionais, e sempre que uma melhoria não prevista se mostrava pertinente durante a implementação, ela era incorporada naquele momento, em vez de ser postergada para um planejamento posterior. Essa incorporação contínua da mudança, e não a cadência fixa de entregas, é o traço do método que efetivamente governou o projeto (BECK; ANDRES, 2004; WILDT et al., 2015).

Quanto à divisão de trabalho, este TCC concentra a atuação de Victor no núcleo de tradução e no back-end, enquanto o texto complementar de Igor trata do front-end. A colaboração manteve a propriedade coletiva do código: ambos os autores atuaram no levantamento de requisitos, no projeto, na implementação, no teste e na revisão, podendo alterar diferentes partes do sistema, com alterações registradas no histórico do repositório (WILDT et al., 2015). As demais práticas foram adotadas nos termos descritos ao longo deste capítulo: o uso de testes

automatizados, detalhado na Seção 4.6; o versionamento e o fluxo de implantação; o *design* simples e a refatoração contínua, que orientaram a organização descrita na Seção 4.1.

O restante deste capítulo está organizado da seguinte maneira. A Seção 4.1 descreve a modelagem da arquitetura do projeto; a Seção 4.2 especifica a linguagem e seus elementos configuráveis; a Seção 4.3 detalha o núcleo do interpretador, fase a fase; a Seção 4.4 trata da integração entre o núcleo e a plataforma web; a Seção 4.5 apresenta os contratos de resposta e os controles do back-end; e a Seção 4.6 expõe a estratégia de testes automatizados.

No ciclo de desenvolvimento guiado por testes tomado como referência, a escrita do teste precede a implementação da funcionalidade: primeiro se especifica o comportamento esperado em um teste que falha, depois se implementa o código necessário para fazê-lo passar e, por fim, se refatora preservando a aprovação da suíte. A integração de uma etapa à seguinte é uma verificação adicional, não o que caracteriza o TDD, conforme a prática consolidada por Aniche (2012) e Martin (2011). Os testes permitem verificar regressões nas configurações da linguagem; sua presença no repositório não comprova, isoladamente, a ordem teste–implementação–refatoração em cada incremento.

4.1 Modelagem da Arquitetura do Projeto

O sistema está organizado em três unidades de implementação no mesmo repositório: `packages/compiler`, que contém o núcleo de tradução e execução em TypeScript; `packages/ide`, que contém a aplicação Next.js e a interface React; e `backend`, que contém a API de persistência em Python, com FastAPI, SQLAlchemy assíncrono e PostgreSQL. A distinção entre regras de tradução, apresentação e persistência segue o princípio de separação de responsabilidades discutido por Martin (2019). O pacote do núcleo é consumido pela IDE como dependência local e pode ser exercitado diretamente pelos testes, sem depender de uma conexão com o banco de dados.

Cabe explicitar quais padrões de projeto sustentam essa organização. O reconhecimento léxico emprega o padrão *Factory Method*: a classe `LexerScannerFactory` examina o caractere corrente e devolve o analisador especializado capaz de tratá-lo (*string*, número, comentário, identificador, terminador de instrução ou símbolo), sem que o `Lexer` precise conhecer as classes concretas. Esses analisadores, por sua vez, materializam o padrão *Strategy*: todos derivam da classe abstrata `LexerScanner` e implementam a mesma operação `run`, o

que permite acrescentar novas categorias de *token* sem alterar o laço principal da varredura. Já a travessia da sequência de *tokens* pelo analisador sintático segue o padrão *Iterator*, encapsulado na classe `TokenIterator`, que oferece as operações de avanço, inspeção antecipada e consumo condicional sem expor a estrutura interna da coleção (MARTIN, 2019).

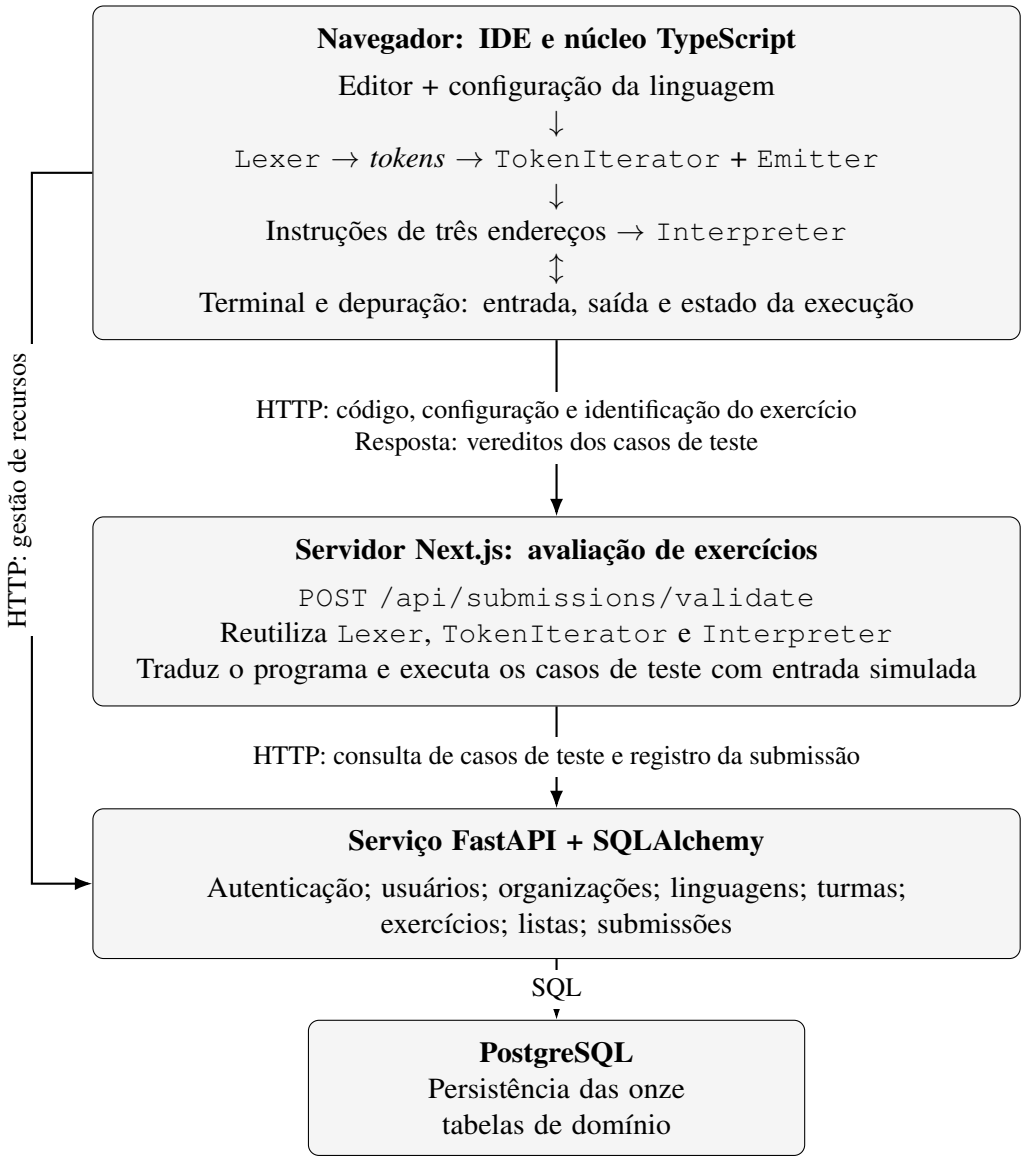
A Figura 4.1 distingue os locais de execução. No navegador, os *hooks* `useLexerAnalyse` e `useIntermediatorCode` instanciam diretamente o `Lexer` e o `TokenIterator`. O primeiro produz os *tokens*; o segundo coordena análise sintática, verificações semânticas e emissão de instruções de três endereços. O componente de terminal instancia o `Interpreter`, conectando suas funções de entrada e saída à interface. A depuração também reutiliza o núcleo no navegador, por meio de `useDebugSession`.

O servidor Next.js reutiliza as mesmas classes na rota `POST /api/submissions/validate`, destinada à avaliação automática de exercícios. A rota traduz o código recebido, consulta os casos de teste no serviço FastAPI e executa as instruções com entradas simuladas, comparando a saída com o resultado esperado. Os fluxos interativos de análise e de execução da IDE não dependem dessa rota: utilizam o pacote diretamente no navegador. A API FastAPI cuida de autenticação, linguagens, turmas, exercícios e submissões; ela não implementa as fases de tradução do código do estudante.

4.1.1 Modelagem do Banco de Dados

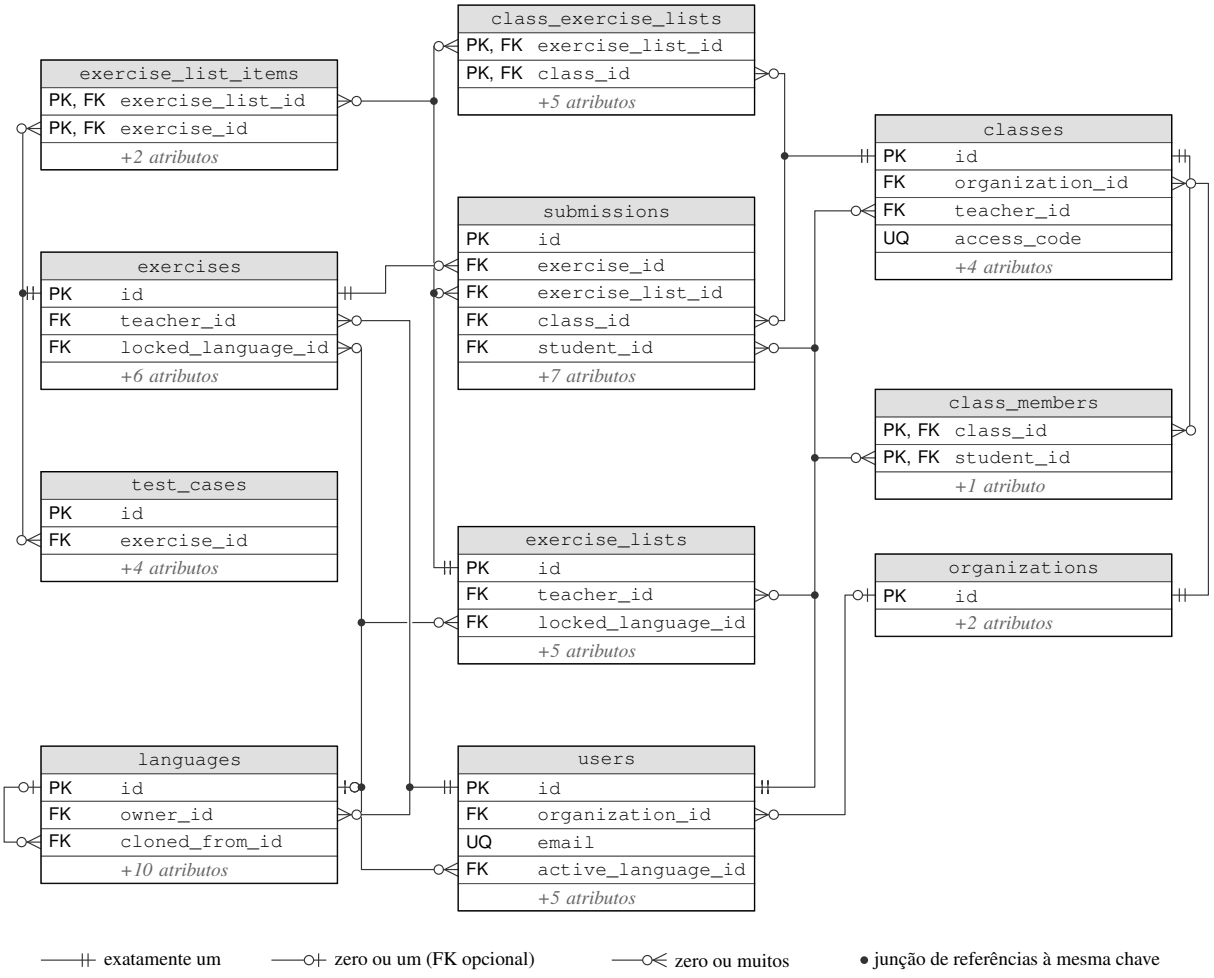
A persistência utiliza PostgreSQL e um esquema relacional mapeado pelas classes de `backend/app/models`. A Figura 4.2 apresenta as onze tabelas de domínio em notação de pé de galinha: cada tabela lista suas colunas de chave, e cada linha liga uma chave estrangeira à tabela referenciada, com a cardinalidade marcada nas extremidades. O Apêndice D completa a figura com os 82 atributos, seus tipos, nulabilidade, chaves, restrições e índices extraídos dos metadados SQLAlchemy. Essa documentação descreve a versão do código-fonte examinada em 19 de outubro de 2026; não constitui uma inspeção de um banco de produção.

Figura 4.1 – Arquitetura e locais de execução do núcleo na plataforma.



Fonte: elaborado pelo autor a partir do código-fonte do projeto.

Figura 4.2 – Tabelas e referências do esquema lógico da plataforma.



Fonte: elaborado pelo autor a partir dos modelos SQLAlchemy; atributos no Apêndice D.

O eixo de acesso reúne `organizations`, `users`, `classes` e `class_members`. Os usuários compartilham uma tabela e são diferenciados pelo campo `role`, cujos valores no modelo são `ADMIN`, `TEACHER`, `STUDENT`, `COMMUNITY` e `SYSTEM`. O vínculo de um usuário com uma organização é opcional no esquema atual. A turma referencia uma organização e um professor; a associação entre aluno e turma é registrada por uma chave primária composta em `class_members`. A existência dessas referências, por si só, não demonstra isolamento de acesso entre organizações, que depende também das regras dos serviços.

A tabela `languages` registra as linguagens configuradas: proprietário, nome, descrição, configuração em JSONB, dados de imagem, identificador de predefinição e informações de publicação. O campo `cloned_from_id` referencia uma linguagem de origem quando há clonagem, e `users.active_language_id` indica a linguagem ativa do usuário. O par proprietário e nome é único. Esses vínculos permitem documentar a persistência da personalização, ausente no esquema apresentado na qualificação.

O conteúdo pedagógico é distribuído entre `exercises`, `test_cases`, `exercise_lists` e `exercise_list_items`. Exercícios e listas possuem política de linguagem `OPEN` ou `LOCKED` e uma referência opcional em `locked_language_id`. Uma restrição de verificação exige que essa referência esteja preenchida exatamente quando a política é `LOCKED`. Os casos de teste pertencem a exercícios e armazenam entrada e saída esperada; os itens de lista associam exercícios a listas, com peso e ordem. O relacionamento de exercícios com casos de teste ordena os resultados por `order_index`. A remoção de casos de teste dependentes está configurada no relacionamento do ORM por `cascade="all, delete-orphan"`, o que deve ser distinguido de uma cláusula SQL `ON DELETE CASCADE`.

A publicação de uma lista para uma turma é registrada em `class_exercise_lists`, cuja chave primária combina os identificadores da lista e da turma. A tabela armazena prazo, nota total, quantidade mínima de exercícios e instantes de publicação e atualização. Já `submissions` referencia exercício, lista, turma e estudante e conserva o código e a configuração de linguagem da tentativa nos campos `code_snapshot` e `language_snapshot`. Seus demais atributos registram o estado da submissão, a pontuação, o comentário do professor e a data de envio. Os valores de estado representam o ciclo de submissão e avaliação. O resultado de cada caso de teste fica no campo `test_case_results`, uma lista em JSON acrescentada pela migração `0a1b2c3d4e5f`, com valor padrão vazio. Cada item guarda o rótulo do caso, a

entrada, a saída esperada, a saída obtida e a indicação de aprovação, conforme o tipo `TTest-CaseResult` da rota de validação. O serviço Python armazena essa lista sem validar sua estrutura.

A separação entre entidades e tabelas associativas segue os conceitos de modelagem relacional discutidos por Elmasri e Navathe (2015). Os instantâneos preservam o contexto de cada tentativa mesmo quando o código ou a linguagem ativa são modificados posteriormente. O relacionamento ORM entre submissão e publicação utiliza os identificadores de lista e turma; o modelo não declara uma chave estrangeira composta de `submissions` para `class_exercise_lists`. Essa distinção está registrada no dicionário para não atribuir ao banco uma restrição que existe apenas na navegação do ORM. O Alembic gerencia as migrações e mantém a tabela técnica `alembic_version`, separada das onze tabelas de domínio.

4.1.2 Especificação das Rotas da API

A interface de comunicação entre os clientes e os serviços da plataforma envolve dois modelos distintos de invocação do compilador, combinados com um conjunto de rotas HTTP expostas pelo serviço de persistência. Uma decisão central de arquitetura foi a de que as fases de análise léxica e de geração de código intermediário, cujo resultado é exibido interativamente na IDE, são executadas inteiramente no navegador do usuário, sem qualquer requisição HTTP. Os *hooks* `useLexerAnalyse` e `useIntermediatorCode` importam e instanciam diretamente as classes `Lexer` e `TokenIterator` do pacote `compiler`, o que é viável porque esse pacote é escrito em TypeScript puro, sem dependências exclusivas de Node.js para essas operações. Essa abordagem elimina a latência de rede nas interações pedagógicas mais frequentes e mantém o servidor da IDE desonerado das requisições de análise.

A única rota HTTP do pacote `ide` diretamente relacionada ao compilador é `POST /api/submissions/validate`. Essa rota, executada no processo do servidor Next.js, recebe o código-fonte e a configuração corrente da linguagem, executa o ciclo completo de análise léxica, geração de código intermediário e interpretação, e avalia o resultado contra os casos de teste do exercício recuperados do serviço de persistência. A rota centraliza a consulta aos casos de teste e o envio da submissão ao serviço de persistência, propagando a credencial recebida nas chamadas HTTP. A implementação inclui um temporizador de cinco segundos por caso, combinado à promessa de execução por `Promise.race`; esse mecanismo não deve ser confundido com isolamento da execução em um processo separado. Ao término da execução,

a rota envia os resultados consolidados ao serviço FastAPI e devolve ao *front-end* a estrutura `TValidationResult`, contendo o status de cada caso de teste, a pontuação obtida e eventuais avisos de compilação.

As demais rotas HTTP da plataforma pertencem ao serviço de persistência, implementado em Python com o *framework* FastAPI e o ORM SQLAlchemy assíncrono. A documentação completa e interativa dessas rotas é gerada automaticamente pelo próprio *framework*, conforme descrito por FASTAPI (2026), e está disponível nos endereços <https://api.dynamicinterpreter.com.br/docs> (interface Swagger UI) e <https://api.dynamicinterpreter.com.br/redoc> (interface ReDoc). Essas rotas organizam-se nos seguintes grupos de recursos:

- **Autenticação** (`/auth`): gerencia o registro e o *login* de usuários, emitindo tokens JWT utilizados nas demais requisições protegidas;
- **Linguagens** (`/languages`): gerencia configurações nomeadas, publicação no catálogo e importação de linguagens;
- **Organizações** (`/organizations`): gerencia as instituições de ensino cadastradas na plataforma, às quais professores e alunos são vinculados;
- **Usuários** (`/users`): expõe operações de consulta e atualização de perfil, com controle de acesso baseado no atributo `role`;
- **Turmas** (`/classes`): permite ao professor criar, editar, encerrar e arquivar turmas; a matrícula de estudantes é realizada por meio do código de acesso único da turma, gerenciado pela sub-rota `/classes/join`;
- **Exercícios** (`/exercises`): centraliza a criação e a manutenção dos enunciados de exercícios, incluindo seus casos de teste, com restrição de escrita ao professor responsável;
- **Listas de exercícios** (`/exercise-lists`): gerencia a composição e a ordenação dos exercícios em uma lista, com controle de pesos de nota por item;
- **Publicação de listas** (`/class-exercise-lists`): controla a associação de uma lista a uma turma, registrando prazo, nota total e mínimo de exercícios exigidos;
- **Submissões** (`/submissions`): armazena e recupera os registros de submissão dos estudantes, incluindo o instantâneo do código, a pontuação e a retroalimentação do professor.

Na versão examinada, `POST /exercises` recebe também os casos de teste, e `PUT /exercises/{id}` substitui os dados do exercício e sua coleção ordenada de testes. A atualização parcial por `PATCH` permanece distinta da substituição completa. A listagem `GET /classes` retorna dados resumidos do professor e contagens de membros e listas. Esses contratos apoiam os fluxos da IDE sem alterar o número de tabelas e atributos do esquema.

Essa organização reflete diretamente a divisão arquitetural do projeto: o compilador opera localmente no navegador para as interações pedagógicas de baixa latência, em uma rota de servidor para a avaliação controlada de submissões, e o serviço FastAPI permanece inteiramente agnóstico em relação ao compilador, concentrando-se exclusivamente na gestão das entidades de domínio pedagógico. A relação completa dos grupos de recursos da API está reproduzida no Apêndice B.

4.2 Especificação da Linguagem e de seus Elementos Configuráveis

A linguagem suportada pelo interpretador foi projetada como uma linguagem imperativa didática, com variantes de tipagem configuráveis, suficientemente expressiva para abranger os conteúdos típicos das disciplinas introdutórias de programação, e simultaneamente flexível para que professores e alunos possam customizar suas regras superficiais. O conjunto base de funcionalidades contempla declarações com ou sem tipo explícito, atribuições, incremento e decremento de variáveis (++ e −), expressões aritméticas, lógicas, relacionais e condicionais ternárias, estruturas condicionais (`if`/`else` e `switch`), estruturas de repetição (`for` e `while`), arranjos uni e multidimensionais com modos de alocação fixos e dinâmicos, e operações de entrada e saída padrão (`print` e `scan`).

A configuração foi motivada pela investigação de escolhas sintáticas no ensino de programação (STEFIK; SIEBERT, 2013). No núcleo implementado, `LexerConfig` reúne os mapeamentos de palavras-chave, operadores, literais booleanos, delimitadores e terminador, além da opção de indentação. Essas escolhas alteram a forma de reconhecer elementos cuja interpretação interna já está definida. A Tabela 4.1 apresenta as possibilidades de redefinição lexical e suas restrições.

O tipo `GrammarConfig`, recebido por `TokenIterator`, reúne quatro seleções: terminadores obrigatórios ou opcionais no fim da linha; blocos delimitados ou por indentação; modo com declaração explícita de tipo ou modo sem essa declaração; e arranjos fixos ou

dinâmicos. Essas opções selecionam alternativas já codificadas no analisador e nos descritores semânticos. A palavra dinâmico no título não equivale especificamente ao modo de tipagem: refere-se ao recebimento dessa configuração para um novo processamento, sem a geração de um analisador arbitrário pelo usuário.

No modo sem declaração explícita de tipo, as palavras canônicas são `variable` e `function`; `variavel` e `funcao` passam a ser exemplos de personalização. A IDE migra as chaves antigas das configurações armazenadas para os nomes atuais. O léxico admite `ç` e `Ç` em identificadores e nomes configuráveis, além das letras latinas sem acento e do sublinhado, com dígitos permitidos após o primeiro caractere. Esse conjunto não representa suporte irrestrito a Unicode.

A IDE disponibiliza quatro configurações prontas: Portugol, Python-like, C-like e Ruby-like. Portugol, a opção inicial na ausência de configuração salva aplicável, utiliza tipos explícitos, arranjos fixos, blocos `inicio`/`fim` e ponto e vírgula obrigatório. C-like combina os mesmos modos com chaves; Python-like combina indentação, declarações sem tipo explícito, arranjos dinâmicos e terminador opcional no fim da linha; Ruby-like utiliza os três últimos modos com blocos `do`/`end`. Esses nomes indicam combinações suportadas pelo núcleo, sem compatibilidade integral com Python, C ou Ruby. Os padrões da IDE também não devem ser confundidos com os valores padrão de `GrammarConfig` quando o núcleo é instanciado diretamente.

O pacote `compiler` passou a incluir, em `language-rules.ts`, um resumo das regras da linguagem dividido em três grupos: regras comuns aos dois modos, regras do modo tipado e regras do modo sem tipo. O resumo cobre, por exemplo, o início da execução pela função `main`, as estruturas de controle aceitas e o comportamento do `switch`, em que um `case` continua no seguinte até encontrar `break`. Na versão examinada, esse arquivo é usado apenas pelo próprio teste. O diálogo de regras exibido na IDE mantém seus textos nos arquivos de idioma da interface, de modo que as duas descrições precisam ser atualizadas em separado.

A validação prévia da configuração é tratada por funções utilitárias dedicadas, como `validateOperatorWordMap`, `validateBooleanLiteralMap`, `validateBlock Delimiters` e `validateStatementTerminatorLexeme`. Cada uma dessas funções verifica restrições léxicas elementares antes do início da análise, impedindo, por exemplo, que palavras-chave personalizadas colidam com identificadores válidos ou que delimitadores

Tabela 4.1 – Elementos da linguagem passíveis de redefinição pelo usuário.

Elemento	**Forma canônica**	**Exemplo de personalização**	**Restrições verificadas**
Palavras-chave de tipo	`int`, `float`, `bool`, `string`	`inteiro`, `real`, `logico`, `texto`	Devem ter forma de identificador e não colidir entre si nem com os demais elementos configurados
Controle de fluxo	`if`, `else`, `for`, `while`, `break`, `continue`, `return`	`se`, `senao`, `para`, `enquanto`, `pare`, `siga`, `retorne`	Mesmas restrições das palavras-chave de tipo
Entrada e saída	`print`, `scan`	`escreva`, `leia`	Mesmas restrições das palavras-chave de tipo
Operadores lógicos	`&&`, `||`, `!`	`e`, `ou`, `nao`	Forma de identificador; sem duplicatas; não podem reusar palavra reservada nem colidir com delimitadores de bloco
Operadores relacionais	`==`, `!=`, `>`, `>=`, `<`, `<=`	`igual`, `diferente`, `maior`, `maiorIgual`, `menor`, `menorIgual`	Mesmas restrições dos operadores lógicos
Literais booleanos	`true`, `false`	`verdadeiro`, `falso`	Forma de identificador; sem duplicatas; não podem conflitar com operadores em forma de palavra nem com delimitadores
Delimitadores de bloco	`{` e `}`	`inicio` e `fim`, ou indentação	Ambos preenchidos, sem espaços, distintos entre si e sem reusar palavra reservada
Terminador de instrução	`;`	`ponto` ou ausência de terminador	Não pode ser vazio, conter espaços, reusar o ponto e vírgula ou caracteres de operadores fixos, nem colidir com palavras já definidas na linguagem

Fonte: elaborado pelo autor, a partir das funções de validação do módulo `config.ts` do pacote `compiler`.

de bloco contenham caracteres inválidos. Essa estratégia preventiva atende aos princípios de detecção precoce de erros discutidos por Cooper e Torczon (2012), transformando inconsistências de configuração em mensagens claras antes mesmo da leitura do código-fonte. A Figura 4.3 apresenta a implementação das quatro funções de validação.

Depois da revisão de setembro, `validateBlockDelimiters` deixou de exigir que os delimitadores tenham forma de identificador. A função recusa agora apenas delimitadores com espaços, iguais entre si ou coincidentes com palavras reservadas. A mudança foi necessária para a predefinição livre do assistente, que informa as próprias chaves como delimitadores, e a Figura 4.3c mostra a versão anterior da função. A validação, porém, passou a aceitar mais do que o analisador léxico reconhece. Em um teste manual com `<:` e `:>`, a configuração foi

aceita, mas o *scanner* de símbolos separou cada delimitador em dois *tokens*, < e `:`, e o bloco não foi identificado. Na versão examinada, funcionam as chaves e os delimitadores com forma de identificador, como `inicio` e `fim`.

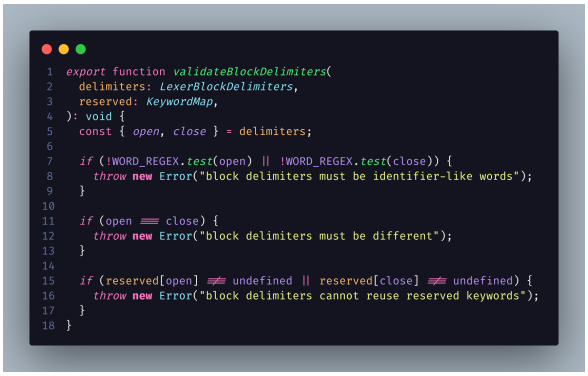
Figura 4.3 – Funções de validação prévia da configuração léxica da linguagem.



(a) `validateOperatorWordMap`



(b) `validateBooleanLiteralMap`



(c) `validateBlockDelimiters`



(d) `validateStatementTerminatorLexeme`

Fonte: elaborado pelo autor.

4.3 Interpretador

O desenvolvimento do interpretador descrito nesta seção, que abrange as fases de análise léxica, sintática e semântica, a geração de código intermediário e a execução do código, constitui o núcleo comum do projeto e foi conduzido de forma colaborativa pelos dois autores, Victor de Souza Vilela da Silva e Igor Lamoia Queiroz, conforme detalhado no Apêndice A. O núcleo de tradução foi implementado em TypeScript, reutilizado no servidor Next.js para avaliar exercícios e no navegador para análise, geração de instruções, execução no terminal e depuração, conforme a Figura 4.1. A escolha pela linguagem TypeScript permitiu o uso intensivo do sistema de tipos durante o desenvolvimento, aproximando o processo de implementação dos benefícios de verificação estática descritos por Aho, Sethi e Ullman (1995) e Martin (2009), e contribuindo para a robustez das estruturas de dados que atravessam todas as fases do compilador.

4.3.1 Análise Léxica

A análise léxica foi implementada na classe `Lexer`, que percorre o código-fonte caractere por caractere e produz uma lista ordenada de instâncias da classe `Token`. Cada *token* carrega quatro atributos essenciais à fase seguinte e à emissão de erros: um identificador numérico (`type`), associado a uma das categorias léxicas suportadas pela linguagem, o lexema literalmente lido do código-fonte, a linha e a coluna em que o lexema iniciou-se. Essa estrutura está alinhada à distinção teórica entre lexema e *token* discutida por Nystrom (2021) e foi mantida estável ao longo de todo o desenvolvimento, pois constitui a interface entre o *front-end* e o restante do compilador. A decomposição de uma instrução em sua sequência de *tokens* segue o conceito ilustrado na Figura 2.2, apresentada no referencial teórico.

Os *tokens* foram organizados em sete grupos temáticos, a saber: aritméticos, lógicos, relacionais, atribuições, palavras reservadas, símbolos e literais, cada um deles definido em um módulo próprio sob o diretório `token/constants`. Essa segregação favorece a manutenção da tabela de *tokens* e habilita o uso de estilos visuais distintos por categoria na IDE, alinhados aos campos `TOKENS_STYLE` associados a cada grupo. A tabela completa de tipos é exportada também na forma invertida (`BY_ID`), o que viabiliza a tradução reversa de identificadores numéricos em descrições legíveis, fundamental para fins de depuração e visualização pedagógica.

A escolha por uma análise léxica determinística orientada por caracteres em vez de geradores automáticos como *Lex* foi motivada pela natureza dinâmica das palavras-chave da

linguagem proposta. Como o vocabulário não é fixo em tempo de compilação, autômatos finitos pré-gerados perderiam a flexibilidade exigida pelo projeto. Em substituição, adotou-se o padrão *Factory* no diretório `lexer/scanners`, no qual a função `LexerScannerFactory.getInstance()` despacha o caractere corrente para o *scanner* apropriado: `comment.ts` para comentários de linha e de bloco, `identifier.ts` para identificadores e palavras-chave, `number.ts` para literais inteiros e em ponto flutuante, `string.ts` para literais de cadeia com sequências de escape, e `symbol-and-operator.ts` para os demais símbolos. Um *scanner* adicional, `statement-terminator.ts`, é instanciado dinamicamente conforme o terminador de instrução configurado pelo usuário, podendo assumir caracteres distintos do ponto e vírgula tradicional ou mesmo ser desativado. A Tabela 4.2 sintetiza os analisadores especializados e suas respectivas responsabilidades.

Tabela 4.2 – Analisadores especializados (*scanners*) despachados pela `LexerScannerFactory`.

Scanner	**Responsabilidade**
`comment.ts`	Comentários de linha (`//`) e de bloco (`/* */`).
`identifier.ts`	Identificadores e palavras-chave, classificados conforme o vocabulário ativo na configuração corrente.
`number.ts`	Literais inteiros e em ponto flutuante.
`string.ts`	Literais de cadeia, incluindo o tratamento de sequências de escape.
`symbol-and-operator.ts`	Operadores e demais símbolos da linguagem.
`statement-terminator.ts`	Terminador de instrução configurável, instanciado dinamicamente conforme a configuração do usuário.

Fonte: elaborado pelo autor.

A função `buildEffectiveKeywordMap`, alimentada pela configuração de personalização proveniente da IDE, constrói em tempo de execução o mapeamento entre os lexemas das palavras-chave correntes e os identificadores numéricos canônicos dos *tokens*. Dessa forma, ao reconhecer um identificador no código-fonte, o analisador léxico verifica primeiro se o lexema corresponde a alguma palavra-chave ativa naquela configuração, classificando-o como `reserved` quando aplicável e como identificador comum caso contrário. Esse mecanismo concentra toda a flexibilidade linguística no analisador léxico, mantendo os estágios subsequentes inalterados ainda que o vocabulário superficial mude entre execuções.

A análise léxica também é responsável pelo modo de delimitação de blocos. Quando a IDE indica que o usuário optou por blocos baseados em indentação, em substituição às chaves, o *lexer* emite *tokens* sintéticos de `indent`, `dedent` e `newline` em pontos apropriados,

aproveitando o registro contínuo de coluna e o tamanho de tabulação configurado pelo usuário. Esse comportamento foi inspirado nas linguagens estruturadas por indentação e permite que a etapa de análise sintática mantenha uma única gramática base, alternando entre blocos delimitados por chaves ou por indentação a partir do conjunto de *tokens* efetivamente gerado.

4.3.2 Análise Sintática

A análise sintática foi implementada por meio de um analisador descendente recursivo, materializado no diretório `grammar/syntax`. A escolha por esta estratégia, em detrimento das famílias *bottom-up* discutidas por Aho, Sethi e Ullman (1995), decorreu de três fatores: a clareza pedagógica da correspondência direta entre regras de produção e funções TypeScript, a facilidade de emissão de mensagens de erro contextualizadas, e a aderência natural ao requisito de visualização passo a passo do processo de compilação dentro da IDE. Cada não-terminal da gramática foi materializado em um arquivo dedicado, como `stmt.ts`, `ifStmt.ts`, `forStmt.ts`, `whileStmt.ts`, `switchStmt.ts`, `declarationStmt.ts`, `ioStmt.ts` e `functionCallExpr.ts`, contendo, em comentários, a derivação formal correspondente.

A navegação sobre a sequência de *tokens* produzida pelo analisador léxico foi encapsulada na classe `TokenIterator`, que oferece operações de `peek`, `next`, `consume` e `match`, além de mecanismos de consulta ao modo de bloco e ao modo de tipagem corrente. O isolamento dessas operações em um único componente padroniza a forma como cada produção avança sobre o fluxo de *tokens* e protege o restante do analisador de possíveis erros de manipulação direta, como leituras fora dos limites ou consumos indevidos. Esta organização materializa o princípio do Padrão Iterador discutido por Silveira et al. (2012).

A gramática base foi cuidadosamente fatorada à esquerda e mantida livre de recursão à esquerda, conforme as restrições necessárias à análise preditiva discutidas por Aho, Sethi e Ullman (1995). Para acomodar as variações configuráveis pelo usuário, a função `consumeStmtTerminator` centraliza a verificação do terminador de instrução, podendo aceitar ponto e vírgula, lexemas alternativos definidos pelo usuário ou mesmo nenhum terminador, conforme a configuração ativa. O mesmo princípio governa o tratamento dos delimitadores de bloco: quando a IDE comunica ao analisador o modo indentado, o reconhecimento de `left_brace` e `right_brace` é substituído pelo casamento dos *tokens* sintéticos `indent` e `dedent` gerados pelo *lexer*, mantendo o restante da gramática inalterado.

A função `exprStmt` reconhece a expressão condicional `condicao ? valorA : valorB` após as expressões lógicas de disjunção, conferindo-lhe menor precedência. As chamadas recursivas permitem condicionais aninhadas. A tradução emite rótulos, um salto condicional e atribuições a uma variável temporária, de modo que apenas o ramo selecionado seja executado. Ramos de mesmo tipo preservam esse tipo; a combinação de inteiro e real resulta em real, a presença de tipo dinâmico resulta em dinâmico e as demais combinações são classificadas como desconhecidas.

Os operadores ++ e − ficam no módulo `updateOperator.ts`. O analisador aceita as formas prefixada e pós-fixada, tanto em uma instrução isolada, como `i++;`, quanto dentro de expressões, o que inclui a atualização de laços como `for (i = 0; i < 3; i++)`. Os dois operadores valem apenas para variáveis escalares: aplicá-los a um elemento de arranjo gera o erro `grammar.invalid_assignment_operator`. No modo tipado, a variável precisa ter sido declarada antes do uso.

4.3.3 Análise Semântica

A análise semântica foi entrelaçada com a fase sintática, na forma de tradução dirigida pela sintaxe descrita por Cooper e Torczon (2012). À medida que o analisador descendente reconhece uma construção válida, ele realiza imediatamente as verificações contextuais correspondentes, antes de prosseguir para a próxima produção. Essa estratégia evita a construção explícita e completa de uma árvore sintática intermediária, reduzindo o uso de memória e simplificando a integração com o gerador de código intermediário descrito na Subseção 4.3.4.

Foram contemplados três grupos principais de verificações semânticas. O primeiro refere-se à resolução de nomes e à validação de escopo: cada identificador citado em uma expressão é confrontado com a tabela de símbolos corrente, e referências a variáveis não declaradas são reportadas como erros semânticos. O segundo grupo trata da consistência de tipos em expressões, atribuições, índices de *arrays* e parâmetros de `scan`; nesse último caso, optou-se pelo registro de um "hint" explícito de tipo (`int` ou `float`) na instrução de entrada, transferindo a coerção final para o interpretador. O terceiro grupo refere-se à aridade e à estrutura: chamadas de funções com número incompatível de argumentos, acessos a *arrays* com número de dimensões diferente do declarado e usos de `break` ou `continue` fora de contextos de repetição são todos sinalizados como erros semânticos contextualizados.

A verificação de conversão diferencia avisos de erros de execução. Converter `float` para `int` gera um aviso de possível perda; promover `int` para `float` é permitido. Outras incompatibilidades conhecidas geram o aviso `grammar.incompatible_type_conversion`; os tipos dinâmico e desconhecido adiam essa verificação. Na execução, `coerceCheckedValue` rejeita, por exemplo, atribuir uma cadeia a uma variável inteira ou retornar um booleano de uma função inteira, emitindo os códigos `interpreter.incompatible_assignment` ou `interpreter.incompatible_return`. Assim, nem toda incompatibilidade interrompe a tradução: alguns casos são avisados antes e rejeitados ao executar a instrução.

4.3.4 Geração de Código Intermediário

A representação intermediária adotada é uma variante de código de três endereços, conforme caracterizado por Aho, Sethi e Ullman (1995) e Cooper e Torczon (2012), materializada pela interface `Instruction`, contendo um operador `op`, um destino `result` e dois operandos `operand1` e `operand2`. O conjunto de operadores suportados é definido como uma união de tipos em `interpreter/constants.ts` e inclui as seis operações aritméticas (`+`, `-`, `*`, `/`, `%` e `//`), as três operações lógicas (`||`, `&&` e `!`), as sete operações relacionais, incluindo a igualdade estrita (===), atribuições, declarações simples e de *arrays*, instruções de acesso e atualização de *arrays* (`ARRAY_GET` e `ARRAY_SET`), saltos condicionais e incondicionais (`IF`, `JUMP` e `RETURN`), rótulos (`LABEL`) e chamadas de sistema (`CALL`).

A geração dessas instruções é realizada pela classe `Emitter`, instanciada por cada análise sintática. O *emitter* mantém dois contadores monotônicos responsáveis pela criação de variáveis temporárias (`__temp0`, `__temp1`, ...) e de rótulos (`__label0`, `__label1`, ...), ambos garantidos como únicos no escopo do programa traduzido. As expressões são decompostas em operações binárias elementares, conforme o exemplo clássico apresentado no referencial teórico, e o controle de fluxo das estruturas `if`, `while`, `for` e `switch` é traduzido em sequências de rótulos e saltos condicionais, eliminando a hierarquia das construções de alto nível em favor da representação linear esperada pelo interpretador.

A opção por gerar três endereços, em detrimento de representações mais abstratas como árvores sintáticas anotadas, foi motivada pelo propósito didático da plataforma. A linearidade das instruções emitidas permite que a IDE apresente, na interface, exatamente o fluxo de operações que será executado, tornando visível para o estudante o vínculo entre construções de alto nível

e suas decomposições elementares. Essa exposição direta da representação intermediária é uma característica diferencial da plataforma em relação às ferramentas analisadas nos trabalhos relacionados.

A inclusão do ternário reutiliza essas instruções de controle de fluxo, sem criar uma instrução intermediária exclusiva para a construção. Já a igualdade estrita possui seu próprio operador === e o identificador de *token* 64; o ponto de interrogação usa o identificador 65. Os identificadores anteriores de palavras-chave foram preservados durante a mudança de `variavel`/`funcao` para `variable`/`function`.

O incremento e o decremento também não têm instrução própria. Na forma prefixada, o *emitter* soma ou subtrai o literal 1 em uma temporária e copia o resultado para a variável; o valor da expressão é o novo valor. Na forma pós-fixada, o valor original é copiado antes para outra temporária, que passa a ser o valor da expressão. Assim, depois de `int i = 3; int antes = i-;`, `antes` vale 3 e `i` vale 2, comportamento verificado em `type-semantics.spec.ts`.

A declaração de várias variáveis escalares em uma só instrução também mudou. Um inicializador passa a valer para as variáveis anteriores da mesma lista que ainda não receberam valor, e o *emitter* gera uma atribuição para cada uma delas. Em `bool a, b, c = true;`, as três variáveis recebem `true`. Em C e em Java, apenas `c` seria inicializada, de modo que essa regra é própria da linguagem do projeto. Quando cada variável tem seu próprio inicializador, como em `bool a = true, b = false;`, cada uma recebe o seu valor.

4.3.5 Interpretação e Tratamento de Erros em Tempo de Execução

A execução das instruções de três endereços foi implementada na classe `Interpreter`, projetada para operar sem pilha explícita de execução em sua versão base, com substituição direta de identificadores por seus valores correntes em uma tabela de símbolos. O ponteiro de instruções avança linearmente sobre a lista de `Instruction`, com exceção dos saltos, que reposicionam o ponteiro para o índice do rótulo associado. Para suportar funções, foi adicionada uma estrutura auxiliar de quadros de chamada (`CallFrame`), que armazena o endereço de retorno, o nome da variável destino e o escopo local da função invocada, permitindo a execução de chamadas e retornos sem comprometer a simplicidade da máquina virtual.

As operações de entrada e saída foram abstraídas por meio de funções de retorno

(`stdout` e `stdin`) passadas ao construtor do interpretador. Essa decisão de projeto isola o motor de execução de qualquer detalhe de apresentação, permitindo que, na IDE, as saídas sejam direcionadas ao componente de terminal embutido, enquanto, nos testes automatizados, as mesmas operações sejam capturadas em *buffers* de *string*. Em conformidade com os princípios de baixo acoplamento e alta coesão discutidos por Martin (2019), essa abstração permitiu reutilizar o mesmo interpretador tanto em modo interativo quanto em modo de validação automatizada.

A leitura de valores via `scan` concentra um dos pontos mais delicados do tratamento de erros em tempo de execução. Como a entrada do usuário pode divergir do tipo esperado pela variável de destino (por exemplo, um valor textual digitado em uma leitura de inteiro), foi implementada uma rotina de coerção, `coerceValueForType`, complementada pela função `parseScanInput`, que inspeciona o "hint" semântico anotado na instrução e tenta converter a cadeia lida para o tipo declarado. Quando a conversão é impossível, em vez de propagar uma exceção genérica de *runtime*, o interpretador lança uma instância de `RuntimeError` contendo a posição original do comando no código-fonte e uma mensagem traduzida via o módulo `i18n`, permitindo que a IDE destaque visualmente a linha responsável e exiba o erro no idioma corrente do usuário.

A igualdade comum (==) compara valores com coerção, enquanto === exige também compatibilidade dos tipos identificados em execução. A implementação consulta o tipo declarado quando disponível e classifica o valor nos demais casos. Nos testes, uma cadeia `"1"` e um inteiro `1` são iguais por ==, mas diferentes por ===; uma variável `float` com valor `1.0` também difere estritamente de uma variável `int` com valor `1`. O operador `!=`, traduzido internamente como <>, mantém a comparação de desigualdade estrita entre valores do hospedeiro; portanto, não é sempre o complemento de == quando os tipos diferem. A igualdade estrita e os símbolos do ternário não possuem campos de substituição por palavras em `OperatorWordMap`.

Duas regras de execução foram acrescentadas depois da revisão de setembro. A primeira trata nomes não declarados. Quando um operando não era número, literal nem variável conhecida, o interpretador devolvia o próprio nome como cadeia. Em `if (pronto)`, sem declaração de `pronto`, a condição era avaliada como verdadeira e o bloco executava. Agora a busca falha com o erro `interpreter.variable_not_found`. No modo tipado o problema aparece antes, na análise, porque `assertTypedReadableIdentifier` passou a conferir também as leituras de variáveis; antes, só as atribuições eram verificadas.

A segunda regra é um limite de passos. Cada instrução executada é contada e, ao passar de 100.000, a execução é interrompida com `interpreter.max_instruction_steps_exceeded`, cuja mensagem avisa que o programa pode conter um laço infinito. O valor pode ser alterado pela opção `maxInstructionSteps` do construtor, mas o terminal da IDE, a depuração e a rota de validação usam o padrão. O limite protege a aba do navegador e o servidor de validação contra laços que não terminam, e também interrompe um programa correto que precise de mais passos.

Os erros estáticos detectados durante a compilação foram modelados pelo módulo `issue`, com três categorias distintas: `IssueError` representa falhas que interrompem a tradução, `IssueWarning` representa alertas não fatais coletados durante a análise, e `IssueInfo` acumula mensagens informativas relevantes ao usuário. Toda ocorrência registra obrigatoriamente um código identificador, uma mensagem, a linha e a coluna do problema, permitindo que a IDE marque o trecho exato no editor e ofereça orientações pedagógicas associadas ao código do erro.

4.4 Integração entre a Plataforma e o Interpretador

A integração entre a plataforma web e o núcleo do interpretador foi materializada por meio do consumo direto do pacote `compiler` como dependência local do pacote `ide`, sem qualquer empacotamento intermediário em formato binário ou em servidor remoto separado. As importações realizadas na rota `/api/submissions/validate` acessam diretamente as classes `Lexer`, `TokenIterator` e `Interpreter`, bem como os tipos de configuração léxica, garantindo que qualquer alteração no compilador se propague à IDE sem etapas adicionais de implantação.

O ciclo de submissão compreende quatro passos. Primeiro, `normalizeCompilerConfig` normaliza a configuração recebida da IDE. Em seguida, a rota instancia o analisador léxico e o iterador de *tokens* e gera as instruções intermediárias uma única vez para o código submetido. Depois, consulta os casos de teste do exercício no serviço FastAPI e, para cada caso, cria um interpretador com entrada e saída ligadas a *buffers* controlados. A comparação normaliza as quebras de linha e remove espaços ao final da saída; o resultado de cada caso compõe um `TTestCaseResult`. Por fim, a rota devolve um `TValidationResult` e, quando não se trata de uma simulação com `dryRun`, solicita o registro da submissão no serviço de persistência.

A persistência de exercícios, turmas e submissões foi mantida em um serviço externo, implementado em Python sobre o *framework* FastAPI e o ORM SQLAlchemy assíncrono, com versionamento de *schema* via Alembic, em conformidade com os princípios de abstração de persistência discutidos por Martin (2019). Essa separação garante que o núcleo de compilação e interpretação permaneça agnóstico em relação à camada de armazenamento, podendo ser executado, inclusive, sem qualquer dependência de banco de dados nas execuções locais e nos testes automatizados.

Nos exercícios com linguagem obrigatória, a interface aplica temporariamente a configuração efetiva resolvida pela API, com precedência do exercício sobre a lista e sobre a seleção livre. A sobreposição bloqueia a seleção de outra linguagem e preserva a configuração pessoal para restauração. O servidor de validação continua recebendo o código e a configuração pelo contrato de submissão; o bloqueio do seletor é um comportamento da interface, não uma demonstração de validação de todas as restrições no servidor.

4.5 Contratos de Resposta e Controles do Back-end

A API separa as entidades persistidas dos modelos Pydantic usados nas requisições e respostas. As rotas declaram modelos de saída, enquanto os serviços aplicam filtros e regras de acesso. A Tabela 4.3 apresenta exemplos identificados na revisão do código. Esses contratos concretizam a seleção de propriedades discutida na Seção 2.2.7; sua existência não demonstra que todos os endpoints retornem o menor conjunto possível de dados (QUEIROZ; SILVA, 2026b; FastAPI, 2026a).

Tabela 4.3 – Exemplos de seleção de dados nas respostas do back-end.

Contrato	**Dados retornados**	**Delimitação**
Usuário	Identificação, organização, perfil e dados de apresentação.	O modelo de saída não declara senha; a atualização de perfil aceita nome, avatar e biografia.
Resumo de linguagem	Identificação, autoria, apresentação, publicação e resumo dos modos da linguagem.	Omite a configuração completa e os campos de consulta de imagem e predefinição.
Histórico no exercício	Identificador, estado, nota e data das submissões do solicitante.	Omite código e configuração capturados na submissão; filtra os registros por aluno autenticado.
Resumo de turma	Dados da turma, contagens e identificação resumida do professor.	Permite apresentar quantidades sem devolver as coleções completas de membros e listas.

Fonte: elaboração própria a partir dos modelos e serviços de Queiroz e Silva (2026b).

No histórico associado à consulta de um exercício, o serviço filtra as submissões pelo identificador do solicitante e, quando informado, pelo da lista. Em seguida, a rota transforma cada registro no modelo de resumo. O campo interno foi denominado `submission_history`, com o nome de saída `submissions`, para evitar que a conversão do objeto persistido expanda automaticamente a relação que reúne submissões de todos os alunos. A seleção de registros e a seleção de campos são, portanto, controles complementares nesse fluxo (QUEIROZ; SILVA, 2026b).

Na autenticação, o serviço utiliza bcrypt para gerar e verificar hashes de senha e JWT assinado com HS256, contendo identificação e expiração. A dependência de autenticação verifica o token antes de disponibilizar o identificador às rotas protegidas; outra dependência restringe os recursos acadêmicos aos perfis previstos. O cadastro público admite aluno, professor e comunidade, sem permitir os perfis administrativo e de sistema. Essa restrição não comprova a identidade institucional de quem se cadastra como professor (QUEIROZ; SILVA, 2026b).

A versão examinada utiliza bcrypt, enquanto a orientação atual da OWASP prioriza Argon2id para novos sistemas e trata bcrypt como opção para sistemas legados. Assim, o mecanismo existente é registrado como decisão de implementação, acompanhado da necessidade de avaliar sua evolução e sua política de parâmetros; não é apresentado como atendimento integral às recomendações atuais (QUEIROZ; SILVA, 2026b; OWASP Foundation, 2026c).

Os contratos de entrada restringem atributos conforme a operação. Por exemplo, o modelo de atualização de usuário não oferece campos para alterar senha, perfil ou organização.

Nas listagens paginadas de linguagens, o tamanho de página tem limite de 100 registros. Entretanto, a listagem pessoal ainda aceita o caminho sem paginação; no serviço de submissões, a paginação recorta em memória uma coleção previamente carregada. Portanto, a redução do volume transmitido não demonstra, por si só, redução proporcional do trabalho no banco ou proteção completa contra consumo excessivo de recursos (QUEIROZ; SILVA, 2026b; OWASP Foundation, 2023c).

4.5.1 Limites dos controles examinados

A inspeção identificou decisões de autorização que ainda precisam de revisão. Na leitura de usuário por identificador, a rota exige autenticação, mas o serviço consultado não recebe o solicitante para comparar organização ou vínculo. Na leitura individual de submissão, a restrição explícita de propriedade é aplicada ao aluno; a atribuição de nota rejeita esse perfil, mas não verifica nesse caminho o vínculo do professor com a atividade. A seleção correta de campos não resolve essas lacunas de autorização por objeto, cuja análise deve considerar a política de acesso pretendida (QUEIROZ; SILVA, 2026b; OWASP Foundation, 2023a).

Esses achados delimitam a segurança demonstrada pelo trabalho. A revisão compreende inspeção de código e testes funcionais locais, sem constituir teste de intrusão ou auditoria da implantação pública (SILVA, 2026b). A conferência sistemática das permissões entre perfis e instituições, da configuração de transporte e da proteção dos tokens permanece necessária. Os testes de autorização devem contemplar tanto operações permitidas quanto recusadas, conforme a orientação da OWASP (OWASP Foundation, 2026a; OWASP Foundation, 2026d).

4.6 Estratégia de Testes Automatizados

4.6.1 Testes do núcleo de tradução

A verificação automatizada das fases do compilador utiliza, na versão do repositório descrita neste texto, o *framework* Vitest. Sua adoção pode ser conferida no arquivo `packages/compiler/package.json`, que declara a dependência e o comando `test`, e nas importações de `describe`, `it` e `expect` dos arquivos de teste. A suíte está organizada no diretório `packages/compiler/src/tests` e estruturada de modo a refletir a topologia interna do projeto. Os testes foram subdivididos em três camadas: testes de *tokens*, que verificam a estabilidade dos identificadores numéricos e dos agrupamentos por categoria; testes de *lexer*,

que exercitam cada *scanner* especializado em isolamento, abrangendo comentários, literais numéricos, cadeias com escapes, terminadores configuráveis, delimitadores de bloco, operadores em forma de palavra e geração de *tokens* de indentação; e testes de gramática, que validam a aceitação ou rejeição de programas inteiros sob distintas configurações de tipagem, modos de bloco e variantes do comando `switch`.

A adoção do TDD discutida por Aniche (2012) mostrou-se particularmente útil em construções como o terminador de instrução configurável e o modo de blocos por indentação, em que pequenas alterações na lógica do *lexer* poderiam quebrar inadvertidamente programas que antes compilavam corretamente. O ciclo iterativo do TDD, composto pelas etapas de escrita do teste, implementação mínima e refatoração, conforme ilustra a Figura 2.3, é a referência metodológica; a inspeção dos arquivos de teste não demonstra sua aplicação sistemática a cada funcionalidade. Em conformidade com a pirâmide de testes apresentada na Seção 2.2.2, a maior parte da suíte concentra-se no nível unitário, garantindo execuções rápidas e *feedback* imediato, enquanto os testes de gramática operam como testes de integração internos ao compilador, verificando o comportamento agregado das fases léxica, sintática e de geração de código intermediário.

Para tornar concreta essa estratégia, o Trecho de Código 4.1 reproduz um dos casos que verificam a aplicação de um mapeamento configurável de operadores no analisador léxico. O teste instancia o `Lexer` com um mapeamento que substitui os operadores lógicos por palavras e verifica que os *tokens* produzidos carregam exatamente os mesmos identificadores de tipo que seriam gerados pelos símbolos canônicos. Essa equivalência é o que garante que as fases seguintes do compilador permaneçam indiferentes à personalização escolhida pelo usuário.

Na verificação local de 19 de outubro de 2026, a suíte do núcleo aprovou 243 testes em 22 arquivos (SILVA, 2026a). Em 19 de setembro eram 224 testes em 21 arquivos (SILVA, 2026b). Os casos acrescentados cobrem incremento e decremento, declarações múltiplas com inicializador no fim da lista, leitura de variáveis não declaradas, o limite de passos, delimitadores informados como símbolos, literais numéricos no fim do código-fonte e o resumo de regras em `language-rules.spec.ts`. Os casos incluem também igualdade comum e estrita, ternários aninhados com execução somente do ramo selecionado, avisos de conversão e rejeição de atribuições e retornos incompatíveis. O Apêndice C apresenta a distribuição e o comando utilizado. Os resultados documentam os comportamentos cobertos, sem constituir prova de correção de todos os programas aceitos.

```
describe("operator word aliases", () => {
  it("tokenizes logical aliases with the same token IDs as
     symbols",
     () => {
    const lexer = new Lexer("if a and not b {}", {
      operatorWordMap: {
        logical_and: "and",
        logical_not: "not",
      },
    });

    const tokens = lexer.scanTokens();

    expect(tokens.map((token) => token.type)).toEqual(
      expect.arrayContaining([
        TOKENS.LOGICALS.logical_and,
        TOKENS.LOGICALS.logical_not,
      ]),
    );
  });
});
```

Listing 4.1 – Caso de teste da equivalência entre operadores simbólicos e operadores em forma de palavra (`operator-word-aliases.spec.ts`).

4.6.2 Testes do serviço FastAPI

A suíte do serviço Python utiliza pytest e pytest-asyncio, com modo assíncrono automático definido na configuração do projeto. Os cenários combinam verificações de esquemas com testes das rotas e de sua integração com serviços e persistência. As *fixtures* centralizam a preparação da aplicação e do banco, e as fábricas de dados criam usuários, organizações e atividades necessários a cada cenário. Essa organização corresponde aos mecanismos de preparação reutilizável apresentados na Seção 2.2.6 (QUEIROZ; SILVA, 2026c; pytest development team, 2026; pytest-asyncio contributors, 2026).

O ambiente emprega SQLite em memória e sessões assíncronas do SQLAlchemy. Uma fixture cria as tabelas a partir dos modelos e insere a organização e o usuário de sistema; ao final da suíte, remove as tabelas. Os testes que utilizam o banco recebem uma sessão com transação e reversão ao término. A dependência de sessão da aplicação é substituída por essa sessão, e o HTTPX utiliza transporte ASGI para enviar requisições às rotas no próprio processo. Assim, as verificações observam respostas HTTP e efeitos no banco de teste, sem depender do serviço PostgreSQL da implantação (QUEIROZ; SILVA, 2026c; FastAPI, 2026a; FastAPI, 2026b).

Entre os casos relacionados à segurança estão o cadastro com retorno dos dados do usuário sem senha, a recusa de credenciais inválidas, a restrição de acesso à submissão de outro aluno e a impossibilidade de um aluno atribuir notas. Também há verificações de ausência de propriedades nas respostas resumidas: os testes de apresentação de linguagens conferem a omissão da configuração completa, enquanto os testes do histórico do exercício verificam que códigos enviados e submissões de colegas não sejam expostos naquele retorno (QUEIROZ; SILVA, 2026c).

O Trecho de Código 4.2 reproduz parte de um caso existente. A preparação da turma, dos alunos e das submissões foi omitida da listagem e está disponível no arquivo original. A asserção do estado HTTP precede a inspeção do conteúdo, de modo que uma resposta de erro sem os campos verificados não seja confundida com um resultado correto (QUEIROZ; SILVA, 2026c).

```
response = await async_client.get(
    f"/exercises/{exercise.id}?listId={exercise_list.id}",
    headers={"Authorization": f"Bearer {token}"},
)

assert response.status_code == 200, response.text
for submission in response.json()["submissions"]:
    assert "codeSnapshot" not in submission
assert "alice-v1" not in response.text
```

Listing 4.2 – Recorte do teste que verifica a ausência de código no histórico resumido do exercício.

4.6.3 Resultados e alcance da verificação

Em 19 de setembro de 2026, a execução local da suíte completa do back-end aprovou 188 casos em 17 arquivos, sem falhas ou casos ignorados, em aproximadamente 59 segundos. Foram utilizados Python 3.12.12, pytest 9.0.2, pytest-asyncio 1.3.0, HTTPX 0.28.1 e FastAPI 0.135.1. A Tabela 4.4 agrupa os arquivos por finalidade, contando separadamente os casos parametrizados coletados pelo executor. O registro estruturado da revisão apresenta a distribuição por arquivo e os comandos utilizados (SILVA, 2026b).

Tabela 4.4 – Distribuição dos testes executados no back-end.

Grupo	Arquivos	Casos	Exemplos de verificação
Autenticação, usuários e organizações	3	29	Cadastro, login, identificação e operações de perfil.
Turmas, exercícios e listas	3	44	Criação, edição, participação e organização de atividades.
Políticas de linguagem e registros de configuração	2	39	Precedência da linguagem obrigatória e configuração preservada na submissão.
Linguagens	4	44	Operações do acervo, esquemas, apresentação e carga inicial.
Submissões e histórico próprio	2	15	Envio, reenvio, correção, acesso e campos do resumo.
Paginação	1	4	Páginas, totais, busca e compatibilidade das listagens.
Cenários complementares	2	13	Saúde do serviço, erros de autenticação e operações recusadas.
Total	**17**	**188**	Casos coletados e aprovados na revisão local.

Fonte: elaboração própria a partir de Silva (2026b).

A contagem corresponde a casos executados e não a uma porcentagem de cobertura do código ou das políticas de segurança. Como as tabelas são criadas diretamente pelos modelos e a dependência de sessão é substituída, essa execução não verifica as migrações Alembic, o comportamento específico do PostgreSQL nem o encerramento da transação da dependência de produção. A aprovação dos cenários existentes também não elimina as lacunas identificadas na Seção 4.5.1 (QUEIROZ; SILVA, 2026c; QUEIROZ; SILVA, 2026b; SILVA, 2026b).

4.6.4 Testes no fluxo de integração contínua

O workflow atual executa verificações nas solicitações de integração e nos envios à branch `main`. O job denominado `unit` reúne Vitest para compilador e IDE e pytest para o back-end; seu nome não implica que todos os casos sejam unitários, pois há integração de rotas e persistência. Outro job executa Playwright com serviços locais em Docker. A implantação por SSH depende da aprovação dos dois jobs e só ocorre em envio à `main`. Esse arquivo documenta a condição para deploy, sem demonstrar a configuração das regras de proteção de branch (QUEIROZ; SILVA, 2026a).

Nos merges examinados, a dependência de sessão passou a encerrar a transação antes do envio da resposta, utilizando o escopo de função do FastAPI. A resposta de autenticação também passou a incluir o modelo de usuário, evitando uma consulta imediata adicional para

obter seus dados. Essas alterações relacionam a consistência do serviço ao fluxo observado pela interface; sua presença no código é distinta da evidência produzida pela suíte com SQLite. A documentação do FastAPI descreve o momento de encerramento das dependências com esse escopo (QUEIROZ; SILVA, 2026b; FastAPI, 2026b).

4.7 Protocolo de Avaliação da Solução

Esta seção descreve o instrumento de avaliação da plataforma, chamado de PA-01, e os critérios para interpretar seus resultados. O instrumento é um só e é compartilhado com o texto do coautor: o mesmo formulário vai para o mesmo grupo de respondentes, e cada trabalho analisa os itens que interessam ao seu tema. Aqui interessam os itens sobre o núcleo de tradução descrito neste capítulo. O protocolo ainda não foi aplicado, e nenhum resultado de participantes aparece neste documento.

4.7.1 Quem responde e o que é possível concluir

A avaliação será feita por um painel de especialistas, formado por professores e profissionais de computação. Não é um estudo com os alunos, que são o público-alvo da plataforma. Pede-se ao respondente que avalie se a ferramenta é adequada para um aluno em primeiro contato com programação, e não que relate a própria experiência de aprendizagem.

Essa escolha limita o que o trabalho pode afirmar. Com um painel de especialistas é possível dizer que profissionais da área acham plausível que a personalização da sintaxe reduza a barreira enfrentada pelo iniciante. Não é possível dizer que ela reduz. Verificar isso exige medir o desempenho de estudantes, o que está registrado na Subseção 4.7.6 como etapa seguinte.

Para o tema deste trabalho, porém, o painel dá o que um aluno iniciante não teria como dar: uma avaliação técnica sobre se o subconjunto implementado da linguagem é suficiente, se a representação intermediária exibida corresponde à execução real e se as mensagens de erro continuam claras depois que o usuário troca o vocabulário da linguagem.

4.7.2 Composição do instrumento

O formulário é respondido pelo próprio participante, distribuído pela internet e criado por um programa, para que sua estrutura possa ser gerada de novo sempre que necessário. São dez páginas, oito delas com grades de itens em escala, como mostra a Tabela 4.5. Três blocos

vêm de instrumentos já consolidados na literatura de avaliação de interfaces; os outros foram escritos para este projeto, porque nenhum instrumento pronto pergunta sobre o que se investiga aqui.

Tabela 4.5 – Blocos do instrumento PA-01, procedência e número de itens.

Bloco	**Procedência**	**Itens**
Consentimento livre e esclarecido	Elaborado para o projeto	1
Perfil profissional	Elaborado para o projeto	4
Uso em sala (condicional)	Elaborado para o projeto	8
Como a plataforma foi usada	Elaborado para o projeto	4
Avaliação dos recursos	Elaborado para o projeto	10
Facilidade de uso	SUS (BROOKE, 1996)	10
Impressão geral	UEQ-S (SCHREPP; HINDERKS; THOMASCHEWSKI, 2017)	8
Potencial pedagógico	Elaborado para o projeto	8
Riscos e limitações	Elaborado para o projeto	4
Qualidade técnica	Elaborado para o projeto	7
Utilidade e intenção de uso	TAM (DAVIS, 1989)	6
Comentários abertos	Elaborado para o projeto	5

Fonte: elaborado pelo autor.

Os instrumentos já conhecidos foram usados porque têm valores de referência publicados. Com eles é possível comparar o escore obtido com o de outros produtos avaliados, em vez de apenas descrever a opinião dos respondentes. Antes do bloco de avaliação dos recursos, o formulário mostra o mesmo programa em duas versões: uma na configuração padrão, com `int`, `for`, `if` e delimitadores simbólicos, e outra personalizada, com `numero_inteiro`, `para`, `se`, operadores escritos por extenso e os delimitadores `inicio` e `fim`. O par mostra, em um caso concreto, a equivalência entre vocabulários diferentes que o núcleo de tradução garante.

4.7.3 Itens sobre o núcleo de tradução

O bloco de qualidade técnica é o que trata mais de perto do tema deste trabalho. Ele é dirigido a quem tem formação na área e traz a opção “não sei”. As afirmações são:

- o subconjunto da linguagem, com tipos, condicionais, repetição, arranjos, funções e entrada e saída, é suficiente para uma disciplina introdutória;
- as mensagens de erro são claras e indicam o que corrigir;
- a ferramenta respondeu de forma rápida e estável;

- o assistente de criação de linguagem é compreensível para um professor sem formação em compiladores;
- a validação prévia impede configurações inválidas;
- o código intermediário exibido corresponde ao que o programa realmente executa.

Esse último item merece atenção. Ele coloca em avaliação externa uma propriedade que o texto sustenta pela forma como o sistema foi construído: a representação intermediária mostrada ao usuário é a mesma que o interpretador consome, e não uma ilustração feita à parte. Uma avaliação negativa nesse item indicaria diferença entre o que a interface exibe e o que a máquina executa, o que comprometeria a premissa central do trabalho.

O bloco termina com uma questão de múltipla escolha sobre construções que faltam na linguagem, com alternativas para registros, orientação a objetos, tratamento de exceções, manipulação de cadeias mais completa, bibliotecas gráficas e divisão do programa em vários arquivos, além da alternativa que diz não ter faltado nada relevante para o ensino introdutório. Essa questão delimita, com base nas respostas, até onde vai o subconjunto implementado e ajuda a priorizar os trabalhos futuros no núcleo.

Do bloco de avaliação dos recursos interessam três itens, todos sobre a exibição das fases intermediárias da tradução: se a tabela de *tokens* ajuda a entender como a plataforma lê o código, se o código intermediário ajuda a entender o que o programa faz e se a execução passo a passo ajuda a acompanhá-lo. Soma-se o item sobre as mensagens de erro ajudarem a identificar o que corrigir, que mede o resultado prático de adaptar essas mensagens ao vocabulário em uso. O bloco de potencial pedagógico repete os três primeiros sob outro ângulo, perguntando pelo valor didático deles para quem está aprendendo, e acrescenta que mostrar os erros no vocabulário escolhido pelo próprio aluno facilita a correção. Essa repetição é proposital: separa o que o especialista achou útil para si, usando a ferramenta, do que ele considera útil para um iniciante.

4.7.4 Escalas usadas e cálculo dos escores

A facilidade de uso usa a escala de Brooke (1996), com dez afirmações que alternam entre positivas e negativas. Essa alternância faz parte do instrumento e não deve ser corrigida. Para calcular o escore, subtrai-se 1 do valor marcado nos itens ímpares, subtrai-se de 5 o valor marcado nos itens pares, somam-se os resultados e multiplica-se o total por 2,5. O resultado

vai de 0 a 100 e não é uma porcentagem. A média de referência fica perto de 68 pontos, e a interpretação segue Bangor, Kortum e Miller (2009) e Sauro e Lewis (2012). A impressão geral usa a versão curta do questionário de Schrepp, Hinderks e Thomaschewski (2017), com oito pares de adjetivos opostos em sete pontos; subtrai-se 4 do valor bruto e separam-se a dimensão pragmática e a hedônica. A utilidade e a intenção de uso vêm do modelo de Davis (1989).

O bloco de riscos funciona ao contrário dos outros: nele, concordar significa apontar um problema. As afirmações são:

- aprender em uma sintaxe personalizada pode dificultar a migração para uma linguagem de mercado;
- alunos da mesma turma usando sintaxes diferentes dificultaria o trabalho do professor;
- a liberdade de personalizar pode desviar a atenção do raciocínio algorítmico;
- o tempo gasto configurando a linguagem não compensa o ganho de aprendizagem.

Os escores desse bloco serão relatados separadamente e não podem ser somados aos do bloco de potencial pedagógico, porque as duas medidas apontam em sentidos opostos e uma anularia a outra. O bloco foi incluído de propósito: se o instrumento trouxesse apenas afirmações favoráveis, a aprovação obtida diria muito pouco.

4.7.5 Participantes, procedimento e questões éticas

Os participantes são convidados diretamente e não são identificados. O formulário registra titulação, área de atuação, ferramentas de ensino que a pessoa já conhece e tempo de experiência com programação. Em uma avaliação feita por especialistas, esses dados fazem parte do resultado e serão apresentados junto com os escores. Um item pede que o respondente marque o que fez na plataforma e o que chegou a personalizar, o que permite separar a avaliação de quem usou a ferramenta da de quem não experimentou o recurso avaliado.

Sobre o número de participantes: de cinco a oito pessoas já revelam a maior parte dos problemas de usabilidade e são suficientes para analisar as respostas abertas. Para apresentar o escore SUS com intervalo de confiança são necessários vinte respondentes ou mais. Com menos que isso, o resultado será apresentado como indicativo, sem afirmar que existe diferença entre subgrupos.

O instrumento foi escrito como pesquisa de opinião pública com participantes não identificados, situação prevista no parágrafo único do artigo 1º da Resolução CNS n.º 510 de 2016 (SAÚDE, 2016). O termo de consentimento segue a Resolução CNS n.º 466 de 2012 (SAÚDE, 2012). Esse enquadramento não vale só porque os pesquisadores o declararam: ele depende de uma manifestação escrita do comitê de ética da instituição, que ainda não foi obtida. Esta especificação não é uma autorização nem uma aprovação ética, e nenhuma resposta pode ser coletada antes desse encaminhamento.

4.7.6 Limites de interpretação e continuidade

Além do limite já citado sobre o público, três ressalvas afetam a leitura dos resultados. A primeira é sobre o instrumento: no formulário, os pares de adjetivos do UEQ-S estão sempre com o polo negativo à esquerda, enquanto o instrumento original inverte alguns pares para evitar que o respondente concorde por hábito. Se a comparação com os valores de referência for importante para o argumento, essa ordem deve ser conferida antes da coleta. A segunda é sobre a redação em português dos itens do SUS, que precisa ser comparada com uma tradução validada. A terceira é sobre a análise: escores altos dados por especialistas não comprovam aprendizagem nem redução de carga cognitiva.

Investigar a hipótese pedagógica é a etapa seguinte e exige outro desenho de estudo: comparar, na mesma plataforma, o vocabulário padrão e o vocabulário personalizado em tarefas equivalentes de entrada e saída, seleção e repetição. A ordem das condições deve ser alternada entre os participantes, e os enunciados, os casos de teste e os critérios de classificação dos erros precisam ser definidos antes da coleta, separando erros de escrita de falhas de lógica. Com isso seria possível descrever os acertos, o tempo até a primeira execução correta e a frequência de erros em cada condição. Ainda assim, uma diferença nessas medidas não comprova, sozinha, que houve aprendizagem duradoura.

5 RESULTADOS DA AVALIAÇÃO

Este capítulo é reservado à análise dos dados do instrumento descrito na Seção 4.7. Até o fechamento desta revisão a coleta não tinha sido feita: o formulário está escrito e revisado, mas não foi distribuído, e o encaminhamento ao comitê de ética da instituição continua pendente. Por isso não há resultados de participantes aqui.

As seções abaixo definem desde já o que será relatado e com quais critérios. Se essa definição fosse feita depois de ver os dados, seria possível escolher o recorte mais favorável à solução; defini-la agora evita essa escolha. Os critérios de cálculo dos escores estão na Seção 4.7 e não são repetidos aqui.

5.1 Composição do Painel

5.2 Avaliação do Núcleo de Tradução

5.3 Suficiência do Subconjunto da Linguagem

5.4 Mensagens de Erro com Vocabulário Personalizado

5.5 Usabilidade e Experiência

5.6 Potencial Pedagógico e Intenção de Uso

5.7 Riscos e Limitações Apontados pelo Painel

5.8 Respostas Abertas

5.9 Síntese e Confronto com os Objetivos

6 CONSIDERAÇÕES FINAIS

Este capítulo retoma o percurso realizado na revisão para o TCC II, confronta o que foi construído com os objetivos estabelecidos na introdução, explicita as limitações do que se apresenta e delimita os encaminhamentos previstos para a etapa seguinte.

6.1 Retomada dos Objetivos

O objetivo geral consistia em projetar e desenvolver um interpretador dinâmico, integrado a uma IDE web, que permitisse ao usuário personalizar a sintaxe e o vocabulário da linguagem. Ao término desta etapa, o núcleo de tradução encontra-se operante e integrado à plataforma, e os objetivos específicos apresentam a seguinte situação.

O primeiro objetivo, especificar a gramática da linguagem e os elementos passíveis de redefinição em tempo de execução, foi cumprido e está registrado na Seção 4.2 e na Tabela 4.1, que relaciona cada elemento configurável, sua forma canônica e as restrições verificadas antes do início da análise.

O segundo e o terceiro, projetar e implementar os módulos de análise léxica, sintática e semântica com suporte à configuração de vocabulário e à seleção das variantes previstas na implementação, foram cumpridos. As fases encadeiam-se do analisador léxico à execução pelo interpretador, conforme detalhado na Seção 4.3, e a configuração da linguagem é recebida e validada a cada nova compilação.

O quarto, integrar o núcleo à interface web definindo os contratos de comunicação entre os módulos, foi cumprido por meio da organização em *monorepo* descrita na Seção 4.1, que permite à IDE consumir o pacote `compiler` como dependência local.

O quinto, disponibilizar o núcleo em ambiente web e verificar seu comportamento por meio de uma suíte de testes automatizados, foi cumprido no que diz respeito à disponibilização e à cobertura das construções da linguagem, conforme a Seção 4.6.

A revisão de 16 de setembro de 2026 acrescenta ao relato a igualdade estrita, a expressão condicional ternária, a validação de tipos durante a execução e o suporte à cedilha no léxico. A suíte do núcleo aprovou 224 casos em 21 arquivos, incluindo os comportamentos introduzidos. Esse resultado não implica compatibilidade integral com as linguagens que inspiram as predefinições da IDE.

A revisão de 19 de setembro de 2026 acrescenta a descrição dos contratos de resposta e dos controles do serviço Python. A execução local aprovou os 188 testes do back-end em 17 arquivos e confirmou os 224 casos do compilador. Os exemplos de minimização incluem resumos de linguagens, respostas de usuário sem senha e histórico de submissões restrito ao solicitante, com omissão do código enviado (QUEIROZ; SILVA, 2026b; SILVA, 2026b).

A revisão de 19 de outubro de 2026 acrescenta os operadores ++ e −, a regra de inicialização em declarações múltiplas, o erro para leitura de variáveis não declaradas, o limite de 100.000 passos na execução e o registro do resultado de cada caso de teste nas submissões. A suíte do núcleo passou a 243 casos em 22 arquivos, e os 188 testes do back-end continuaram aprovados (SILVA, 2026a).

O sexto objetivo, definir o protocolo de avaliação da solução, foi atendido pela Seção 4.7, que descreve quem responde, os blocos de itens, as escalas usadas e o cálculo dos escores, além dos requisitos já apresentados nas Tabelas 3.1 e 3.2. Para aplicar o instrumento ainda falta definir a amostra e obter o encaminhamento de ética da instituição, e o Capítulo 5 registra essa pendência. Vale separar o que o protocolo permite concluir: como é dirigido a um painel de especialistas, ele coleta a opinião de profissionais sobre a plausibilidade da contribuição pedagógica. Provar essa contribuição exigiria um estudo com estudantes.

6.2 Limitações

Cabe explicitar os limites do que aqui se apresenta, de modo que os resultados não sejam lidos além do que sustentam.

A limitação mais relevante diz respeito à ausência de verificação empírica. A pergunta que orienta a pesquisa indaga em que medida a personalização da sintaxe contribui para reduzir a barreira do primeiro contato com a programação; o que este trabalho entrega é o artefato que torna essa investigação possível, e não a resposta a ela. Nenhuma afirmação sobre ganho de aprendizagem pode ser sustentada com os dados atualmente disponíveis.

No plano técnico, a linguagem suportada é um subconjunto didático de linguagens imperativas, com declaração explícita de tipo ou modo dinâmico. Ela cobre as construções típicas das disciplinas introdutórias, mas não contempla orientação a objetos, tratamento estruturado de exceções nem modularização em múltiplos arquivos de código-fonte.

A personalização abrange o vocabulário, operadores em forma de palavra, literais

booleanos e delimitadores, além da seleção de variantes predefinidas de terminação de instruções, blocos, tipagem e arranjos. O conjunto de produções e as alternativas aceitas pelo analisador são definidos na implementação: o usuário não acrescenta novas produções nem redefine livremente a semântica das operações. Assim, o caráter dinâmico corresponde à aplicação de uma configuração a cada novo processamento do código, dentro dos limites suportados pelo núcleo. Esses limites nem sempre coincidem com o que a validação aceita: delimitadores de bloco formados por símbolos diferentes das chaves passam pela verificação da configuração, mas não são reconhecidos pelo analisador léxico, como descrito na Seção 4.2.

No serviço Python, os testes utilizam SQLite em memória e não verificam migrações ou características específicas do PostgreSQL. A inspeção também identificou lacunas de autorização por objeto e caminhos de listagem sem paginação obrigatória, descritos na Seção 4.5.1. A aprovação da suíte e a filtragem de campos nas respostas não constituem garantia de segurança integral (QUEIROZ; SILVA, 2026b; QUEIROZ; SILVA, 2026c; SILVA, 2026b).

Por fim, o interpretador executa o código intermediário por percurso direto das instruções, sem etapa de otimização. As técnicas discutidas no referencial teórico foram estudadas, mas não implementadas, uma vez que o porte dos programas produzidos em contexto introdutório não as torna necessárias.

6.3 Encaminhamentos

A etapa seguinte do Trabalho de Conclusão de Curso concentrará esforços no aprofundamento do núcleo de tradução e na conclusão do protocolo de avaliação. No plano técnico, prevê-se a ampliação da cobertura de testes, sobretudo quanto à robustez do analisador diante das variações configuráveis da linguagem, e a verificação sistemática da correção do código intermediário gerado. No plano metodológico, prevê-se a consolidação dos critérios que permitirão aferir a contribuição da solução.

Para o back-end, os próximos passos incluem revisar as permissões entre usuários e instituições, acrescentar cenários negativos para os vínculos exigidos, limitar as listagens e conferir os controles no ambiente PostgreSQL. A política de credenciais e tokens também requer revisão periódica. Esses encaminhamentos complementam o fluxo já implementado de execução automática dos testes como condição para deploy (OWASP Foundation, 2026a; OWASP Foundation, 2023c; OWASP Foundation, 2026c; QUEIROZ; SILVA, 2026a).

A aplicação desse protocolo junto a estudantes, com a coleta de evidências empíricas em ambiente educacional, é registrada como trabalho futuro. Sua condução exigirá, além do protocolo consolidado, a articulação com turmas em andamento e a observância dos procedimentos éticos aplicáveis a pesquisas que envolvem seres humanos.

Como desdobramentos adicionais, identificam-se a ampliação do subconjunto da linguagem suportado pelo interpretador, a investigação de mecanismos que permitam configurar também produções gramaticais, além dos parâmetros e das variantes predefinidas, e o estudo da geração de código para um alvo executável fora do ambiente da plataforma.

REFERÊNCIAS

ADAPTWORKS. **Agile: Desenvolvimento de Software com Entregas Frequentes e Foco no Valor de Negócio**. São Paulo: Casa do Código, 2012.

AHO, A. V.; SETHI, R.; ULLMAN, J. D. **Compiladores**: princípios, técnicas e ferramentas. Rio de Janeiro: LTC, 1995. Tradução de Daniel de Ariosto Pinto.

ALCANTARA, F. **Tabela de Símbolos**. 2024. Disponível em: <https://frankalcantara.com/lf/11-tabelaSimbolos.html>. Acesso em: 30 mar. 2026.

ANICHE, M. **Test-Driven Development: Teste e Design no Mundo Real**. São Paulo: Casa do Código, 2012.

BANGOR, A.; KORTUM, P.; MILLER, J. Determining what individual sus scores mean: adding an adjective rating scale. **Journal of Usability Studies**, v. 4, n. 3, p. 114–123, 2009.

BECK, K.; ANDRES, C. **Extreme Programming Explained: Embrace Change**. 2. ed. Boston: Addison-Wesley, 2004.

beecrowd. **FAQs Problems**. 2026. Disponível em: <https://judge.beecrowd.com/en/faqs/about/problems>. Acesso em: 09 set. 2026.

BENEDETTI, T. Como escolher a melhor plataforma educativa em 2025. **Blog TutorMundi**, 2025. Disponível em: <https://tutormundi.com/blog/>. Acesso em: 07 abr. 2026.

BROOKE, J. Sus: a quick and dirty usability scale. In: JORDAN, P. W.; THOMAS, B.; WEERDMEESTER, B. A.; MCCLELLAND, I. L. (Ed.). **Usability Evaluation in Industry**. London: Taylor & Francis, 1996. p. 189–194.

COOPER, K. D.; TORCZON, L. **Engineering a Compiler**. 2. ed. Waltham: Elsevier / Morgan Kaufmann, 2012.

DASROOT. **Python Flask vs FastAPI vs Django: Framework Comparison 2026**. 2026. Disponível em: <https://dasroot.net>. Acesso em: 07 abr. 2026.

DAVIS, F. D. Perceived usefulness, perceived ease of use, and user acceptance of information technology. **MIS Quarterly**, v. 13, n. 3, p. 319–340, 1989.

DUNOSSAURO, E. **FastAPI do Zero!** 2026. Disponível em: <https://dunossauro.github.io/fastapi-do-zero/>. Acesso em: 07 abr. 2026.

ELMASRI, R.; NAVATHE, S. B. **Fundamentals of Database Systems**. 7. ed. Hoboken: Pearson, 2015.

FastAPI. **Async Tests**. 2026. Disponível em: <https://fastapi.tiangolo.com/advanced/async-tests/>. Acesso em: 19 set. 2026.

FastAPI. **Dependencies with yield**. 2026. Disponível em: <https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-with-yield/>. Acesso em: 19 set. 2026.

FASTAPI. **Recursos e Documentação Oficial do FastAPI**. 2026. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 07 abr. 2026.

FastAPI. **Response Model: Return Type**. 2026. Disponível em: <https://fastapi.tiangolo.com/tutorial/response-model/>. Acesso em: 19 set. 2026.

FastAPI. **Testing Dependencies with Overrides**. 2026. Disponível em: <https://fastapi.tiangolo.com/advanced/testing-dependencies/>. Acesso em: 19 set. 2026.

FERREIRA, B.; SILVA, C. L.; RIBEIRO, F. d. C. L. Um estudo de caso da ferramenta beecrowd como tecnologia da informação e comunicação na aprendizagem de algoritmos. **ForScience**, v. 13, n. 2, 2025. Disponível em: <https://doi.org/10.29069/forscience.2025v13n2.e1313>. Acesso em: 07 abr. 2026.

FIRMINIQ. **Test-Driven Development (TDD) and its Impact on Quality Assurance (QA)**. 2024. Disponível em: <https://firminiq.com/test-driven-development-tdd-and-its-impact-on-quality-assurance-qa/>. Acesso em: 06 abr. 2026.

GARCIA, L. **Beecrowd: como desenvolver habilidades praticando programação**. Alura, 2024. Disponível em: <https://www.alura.com.br/artigos/beecrowd-como-desenvolver-habilidades-praticando-programacao>. Acesso em: 07 abr. 2026.

GOMES, D. **O que é Portugol Studio?** DevMedia, 2020. Disponível em: <https://www.devmedia.com.br/o-que-e-portugol-studio/40764>. Acesso em: 07 abr. 2026.

LADNER, R. E. **Learn About the New Quorum Studio**. Computer Science Teachers Association (CSTA), 2019. Disponível em: <https://csteachers.org/learn-about-the-new-quorum-studio/>. Acesso em: 07 abr. 2026.

MÃO NO CÓDIGO. **BEECROWD: Todo PROGRAMADOR deveria conhecer | BEECROWD (antigo Uri Online Judge)**. 2022. Vídeo do YouTube. Disponível em: <https://www.youtube.com/watch?v=VzOhnK0CuiE>. Acesso em: 14 ago. 2026.

MARTIN, R. C. **Código Limpo: Habilidades Práticas do Agile Software**. Rio de Janeiro: Alta Books, 2009.

MARTIN, R. C. **The Clean Coder: A Code of Conduct for Professional Programmers**. Upper Saddle River: Prentice Hall, 2011.

MARTIN, R. C. **Arquitetura Limpa: O Guia do Artesão para Estrutura e Design de Software**. Rio de Janeiro: Alta Books, 2019.

MARTINS, C. **Portugol Studio: Guia Completo de Como Usar a Plataforma**. HostGator, 2025. Disponível em: <https://www.hostgator.com.br/blog/portugol-studio/>. Acesso em: 07 abr. 2026.

Mind Makers. 10 tendências em edtech para 2026: o que a escola precisa saber. **Blog Mind Makers**, 2025. Disponível em: <https://mindmakers.com.br/blog/>. Acesso em: 07 abr. 2026.

NYSTROM, R. **Crafting Interpreters**. [S.l.]: Genever Benning, 2021. ISBN 978-0990582939.

OWASP Foundation. **API1:2023 Broken Object Level Authorization**. 2023. Disponível em: <https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/>. Acesso em: 19 set. 2026.

OWASP Foundation. **API3:2023 Broken Object Property Level Authorization**. 2023. Disponível em: <https://api-security.owasp.org/editions/2023/en/0xa3-broken-object-property-level-authorization/>. Acesso em: 19 set. 2026.

OWASP Foundation. **API4:2023 Unrestricted Resource Consumption**. 2023. Disponível em: <https://api-security.owasp.org/editions/2023/en/0xa4-unrestricted-resource-consumption/>. Acesso em: 19 set. 2026.

OWASP Foundation. **Authorization Cheat Sheet**. 2026. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html>. Acesso em: 19 set. 2026.

OWASP Foundation. **Input Validation Cheat Sheet**. 2026. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html>. Acesso em: 19 set. 2026.

OWASP Foundation. **Password Storage Cheat Sheet**. 2026. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html>. Acesso em: 19 set. 2026.

OWASP Foundation. **REST Security Cheat Sheet**. 2026. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html>. Acesso em: 19 set. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma Abordagem Profissional**. 8. ed. Porto Alegre: AMGH, 2016.

PRICE, A. M. A.; TOSCANI, S. S. **Implementação de Linguagens de Programação: Compiladores**. Porto Alegre: Sagra Luzzatto, 2001.

pytest-asyncio contributors. **pytest-asyncio: Concepts**. 2026. Disponível em: <https://pytest-asyncio.readthedocs.io/en/stable/concepts.html>. Acesso em: 19 set. 2026.

pytest development team. **How to use fixtures**. 2026. Disponível em: <https://docs.pytest.org/en/stable/how-to/fixtures.html>. Acesso em: 19 set. 2026.

QUEIROZ, I. L.; SILVA, V. d. S. V. d. **Dynamic Interpreter: fluxo de integração contínua e implantação**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/tree/e0a012bb833fa829e6e694bc2f5b99fdff4863a0/.github/workflows/deploy.yml>. Acesso em: 19 set. 2026.

QUEIROZ, I. L.; SILVA, V. d. S. V. d. **Dynamic Interpreter: implementação do serviço back-end**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/tree/e0a012bb833fa829e6e694bc2f5b99fdff4863a0/backend/app>. Acesso em: 19 set. 2026.

QUEIROZ, I. L.; SILVA, V. d. S. V. d. **Dynamic Interpreter: testes do serviço back-end**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/tree/e0a012bb833fa829e6e694bc2f5b99fdff4863a0/backend/tests>. Acesso em: 19 set. 2026.

QUORUM. **The Quorum Programming Language: Documentation and Curriculum**. 2025. Disponível em: <https://quorumlanguage.com/>. Acesso em: 09 set. 2026.

SAÚDE, B. C. N. de. **Resolução n. 466, de 12 de dezembro de 2012**. Brasília: [s.n.], 2012. Diário Oficial da União.

SAÚDE, B. C. N. de. **Resolução n. 510, de 7 de abril de 2016**. Brasília: [s.n.], 2016. Diário Oficial da União.

SAURO, J.; LEWIS, J. R. **Quantifying the User Experience**: practical statistics for user research. Waltham: Morgan Kaufmann, 2012.

SCHREPP, M.; HINDERKS, A.; THOMASCHEWSKI, J. Design and evaluation of a short version of the user experience questionnaire (ueq-s). **International Journal of Interactive Multimedia and Artificial Intelligence**, v. 4, n. 6, p. 103–108, 2017.

Scratch Foundation. **Getting Started**. 2026. Disponível em: <https://scratchfoundation.org/learn/learning-library/getting-started>. Acesso em: 09 set. 2026.

SILVA, M. L. d.; PEREIRA, S. S. L.; LIMA, D. R. **Laila: um gerador de analisador léxico e sintático online baseado em FLEX e Bison**. 2021. Instituto Federal do Ceará, Campus Aracati. Disponível em: <https://gestaoaracati.ifce.edu.br/attachments/download/2699/tcc.pdf>. Acesso em: 09 set. 2026.

SILVA, V. d. S. V. d. **Registro de verificação local dos testes do back-end e do compilador: 19 de outubro de 2026**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/blob/escrita-tcc/qualificacao/victor/verificacao/testes-2026-10-19.json>. Acesso em: 19 out. 2026.

SILVA, V. d. S. V. d. **Registro de verificação local dos testes do back-end e do compilador: 19 de setembro de 2026**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/blob/escrita-tcc/qualificacao/victor/verificacao/testes-backend-2026-09-19.json>. Acesso em: 19 set. 2026.

SILVEIRA, P.; SILVEIRA, G.; LOPES, S.; MOREIRA, G.; STEPPAT, N.; KUNG, F.; WEBBER, J. **Introdução à arquitetura e design de software: uma visão sobre a plataforma Java**. São Paulo: Casa do Código, 2012.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SOUZA, D. M.; BATISTA, M. H. d. S.; BARBOSA, E. F. Problemas e dificuldades no ensino e na aprendizagem de programação: um mapeamento sistemático. **Revista Brasileira de Informática na Educação**, v. 24, n. 1, p. 39–52, 2016.

STEFIK, A.; SIEBERT, S. An empirical investigation into programming language syntax. **ACM Transactions on Computing Education**, ACM, v. 13, n. 4, 2013.

UEYAMA, R. **chibicc: A small C compiler**. 2020. Disponível em: <https://github.com/rui314/chibicc>. Acesso em: 09 set. 2026.

UNIVALI. **Portugol Studio - LITE**. Laboratório de Inovação Tecnológica na Educação, 2026. Disponível em: <https://univali-lite.github.io/Portugol-Studio/>. Acesso em: 09 set. 2026.

W3C. **Accessible Rich Internet Applications (WAI-ARIA) 1.2**. [S.l.], 2023. Disponível em: <https://www.w3.org/TR/wai-aria-1.2/>. Acesso em: 14 ago. 2026.

WILDT, D.; MOURA, D.; LACERDA, G.; HELM, R. **eXtreme Programming: práticas para o dia a dia no desenvolvimento ágil de software**. São Paulo: Casa do Código, 2015.

A APÊNDICE A – PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES

Este apêndice detalha a divisão de responsabilidades entre os dois autores, conforme a pré-proposta aprovada e o plano de trabalho aprovado pelo Colegiado do Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina.

A.1 Atividades Desenvolvidas em Regime Colaborativo

As atividades listadas a seguir, de natureza estruturante ou situadas na fronteira entre os dois subsistemas, foram conduzidas de forma colaborativa por Victor de Souza Vilela da Silva e Igor Lamoia Queiroz:

- Revisão bibliográfica sobre interpretadores, compiladores e ferramentas educacionais correlatas;
- Definição dos requisitos funcionais e não funcionais e da arquitetura geral do sistema;
- Implementação e validação do núcleo do interpretador, abrangendo as fases de análise léxica, análise sintática, geração de código intermediário e execução, conduzidas conjuntamente pelos dois autores;
- Definição dos contratos de comunicação entre o núcleo do interpretador e a plataforma web e a integração entre os módulos;
- Revisão mútua de código e tomada de decisões arquiteturais nas etapas de refatoração e evolução do sistema.

A.2 Eixos de Aprofundamento Individual

A partir da base técnica construída em conjunto, cada autor aprofundou sua análise acadêmica em eixos específicos, conforme previsto na pré-proposta.

A.2.1 Victor de Souza Vilela da Silva (esta monografia)

- Detalhamento das fases de análise léxica e do sistema de varredura por escâneres especializados (*factory pattern*);

- Descrição aprofundada da análise sintática por descida recursiva e da correspondência entre as produções do analisador e a gramática formal da linguagem Java--;
- Detalhamento e documentação da geração de código de três endereços: instrução a instrução, tabela de temporários e tabela de rótulos;
- Análise aprofundada dos parâmetros de configuração da linguagem, que permitem ao núcleo aplicar o vocabulário escolhido e selecionar variantes de escrita predefinidas a cada nova análise e execução;
- Análise do funcionamento do interpretador, da tabela de símbolos e do despacho das chamadas de sistema de entrada e saída;
- Desenvolvimento do *back-end* da plataforma em FastAPI, incluindo a modelagem das entidades de domínio, a implementação das rotas REST, a persistência com SQLAlchemy assíncrono e o suporte a autenticação, turmas, exercícios, listas e submissões;
- Verificação técnica do núcleo de tradução, aferindo o comportamento do interpretador frente a programas de teste elaborados para cobrir as construções da linguagem.

A.2.2 Igor Lamoia Queiroz (documento complementar)

- Detalhamento da arquitetura da aplicação web (Next.js, Pages Router) e da integração com o Monaco Editor para edição de código com realce de sintaxe dinâmico;
- Documentação do assistente de personalização da linguagem (*wizard*), incluindo o modelo de etapas, a lógica de validação preventiva dos delimitadores de bloco e o mecanismo de pré-visualização em tempo real;
- Análise da experiência de uso da plataforma, dos contextos React que governam o estado da aplicação e das mediações pedagógicas oferecidas pela interface;
- Descrição, sob a ótica da interface, dos fluxos de submissão, correção automatizada de exercícios e gerenciamento de turmas;
- Definição do protocolo de avaliação da experiência de uso, contemplando os instrumentos de coleta de *feedback*, as métricas de usabilidade e os critérios de engajamento a serem aplicados junto aos estudantes em etapa posterior.

B APÊNDICE B – DOCUMENTAÇÃO DA API

Este apêndice reúne a relação dos recursos expostos pela API da plataforma. A documentação completa e interativa é gerada automaticamente pelo *framework* FastAPI e está disponível *online* nas interfaces Swagger UI (<https://api.dynamicinterpreter.com.br/docs>) e ReDoc (<https://api.dynamicinterpreter.com.br/redoc>). A Tabela B.1 sintetiza os grupos de recursos e suas responsabilidades.

Tabela B.1 – Grupos de recursos da API da plataforma.

Recurso	**Responsabilidade**
`POST /api/submissions/validate`	Executa o ciclo completo de compilação e interpretação do código submetido e o avalia contra os casos de teste do exercício (rota do pacote `ide`).
`/auth`	Registro e *login* de usuários, com emissão de *tokens* JWT.
`/languages`	Configurações de linguagem, publicação no catálogo e importação.
`/organizations`	Gestão das instituições de ensino cadastradas.
`/users`	Consulta e atualização de perfil, com controle de acesso por `role`.
`/classes`	Criação, edição, encerramento e arquivamento de turmas; matrícula via `/classes/join`.
`/exercises`	Criação e manutenção de enunciados e respectivos casos de teste.
`/exercise-lists`	Composição, ordenação e atribuição de pesos aos exercícios de uma lista.
`/class-exercise-lists`	Publicação de listas para turmas, com prazo, nota total e mínimo de exercícios.
`/submissions`	Armazenamento e recuperação das submissões dos estudantes, com pontuação e retroalimentação.

Fonte: elaborado pelo autor.

C APÊNDICE C – CASOS DE TESTE AUTOMATIZADOS

Este apêndice resume a suíte de testes automatizados do núcleo do compilador, construída com o *framework* Vitest. A suíte reúne **243 casos** distribuídos em 22 arquivos de teste, organizados de modo a refletir as camadas internas do projeto, conforme a Tabela C.1. Os casos estão disponíveis no repositório do projeto (<https://github.com/igorlamoia/ts-compilator-for-java>), no diretório `packages/compiler/src/tests`. Na execução de verificação realizada em 19 de outubro de 2026, o comando `npx vitest run`, a partir do pacote `compiler`, aprovou os 243 casos, sem falhas. O resultado foi registrado junto à execução do serviço Python (SILVA, 2026a). Em 19 de setembro de 2026 a suíte tinha 224 casos em 21 arquivos (SILVA, 2026b). A distribuição dos testes do serviço Python é apresentada na Tabela 4.4. O total deste apêndice refere-se somente ao compilador e documenta o estado da suíte, sem atribuir a esta execução evidências sobre o que foi demonstrado na apresentação de qualificação.

Tabela C.1 – Distribuição dos casos de teste por camada do compilador.

Camada	**Arquivos**	**Casos**	**Escopo**
Tokens	1	13	Estabilidade dos identificadores numéricos e dos agrupamentos por categoria.
Análise léxica	9	65	*Scanners* em isolamento: comentários, literais numéricos e de cadeia, terminador configurável, delimitadores de bloco, operadores em forma de palavra, geração de *tokens* de indentação e literais numéricos no fim do código-fonte.
Análise sintática e semântica	9	148	Aceitação ou rejeição de programas sob distintos modos de tipagem e de bloco, variantes do `switch`, ternário, igualdade estrita, incremento e decremento, declarações múltiplas e verificação de tipos.
Interpretação	2	15	Execução do código de três endereços, sessões de depuração, variáveis não declaradas e limite de passos.
Regras da linguagem	1	2	Estrutura do resumo exportado por `language-rules.ts`.
Total	**22**	**243**	

Fonte: elaborado pelo autor.

D APÊNDICE D – DICIONÁRIO DE DADOS

Este apêndice documenta os modelos SQLAlchemy de `backend/app/models`, sem consultar um banco em execução. Os tipos são apresentados para o dialeto PostgreSQL. PK identifica chave primária; FK, chave estrangeira; UQ, unicidade; e a coluna Nulo informa se o atributo admite valor nulo. Os valores enumerados, as restrições compostas e os índices declarados são registrados após cada tabela. Uma FK sem ação de exclusão explicitada não equivale a uma cascata de exclusão no banco.

O relacionamento ORM de submissão com publicação utiliza os identificadores de lista e turma, mas não declara uma FK composta correspondente. Cascatas definidas apenas por `relationship` pertencem ao ORM, não às restrições SQL listadas aqui. A tabela técnica `alembic_version`, mantida pelo Alembic, não integra o conjunto de entidades de domínio.

`class_exercise_lists`

Atributo	Tipo	Nulo	Chaves e referências
`exercise_list_id`	`INTEGER`	Não	PK; FK: `exercise_lists.id`; exclusão: `CASCADE`
`class_id`	`INTEGER`	Não	PK; FK: `classes.id`; exclusão: `CASCADE`
`deadline`	`TIMESTAMP WITHOUT TIME ZONE`	Não	–
`total_grade`	`FLOAT`	Não	–
`min_required`	`INTEGER`	Não	–
`published_at`	`TIMESTAMP WITHOUT TIME ZONE`	Não	–
`updated_at`	`TIMESTAMP WITHOUT TIME ZONE`	Não	–

- Chave primária composta: `exercise_list_id`, `class_id`.
- Índice `ix_class_exercise_lists_class_id` sobre `class_id`.

class_members

Atributo	Tipo	Nulo	Chaves e referências
class_id	INTEGER	Não	PK; FK: classes.id
student_id	INTEGER	Não	PK; FK: users.id
joined_at	TIMESTAMP WITHOUT TIME ZONE	Não	–

- Chave primária composta: class_id, student_id.

classes

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
organization_id	INTEGER	Não	FK: organizations.id
teacher_id	INTEGER	Não	FK: users.id
name	VARCHAR	Não	–
description	VARCHAR	Não	–
access_code	VARCHAR	Não	UQ
created_at	TIMESTAMP WITHOUT TIME ZONE	Não	–
status	classstatus	Não	–

- status: ACTIVE, ARCHIVED.

exercise_list_items

Atributo	Tipo	Nulo	Chaves e referências
exercise_list_id	INTEGER	Não	PK; FK: exercise_lists.id; exclusão: CASCADE
exercise_id	INTEGER	Não	PK; FK: exercises.id; exclusão: RESTRICT
grade_weight	FLOAT	Não	–

Atributo	Tipo	Nulo	Chaves e referências
order_index	INTEGER	Não	–

- Chave primária composta: exercise_list_id, exercise_id.
- Índice ix_exercise_list_items_exercise_id sobre exercise_id.

exercise_lists

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
teacher_id	INTEGER	Não	FK: users.id
title	VARCHAR	Não	–
description	VARCHAR	Não	–
language_policy	languagepolicy	Não	–
locked_language_id	INTEGER	Sim	FK: languages.id; exclusão: RESTRICT
created_at	TIMESTAMP WITHOUT TIME ZONE	Não	–
updated_at	TIMESTAMP WITHOUT TIME ZONE	Não	–

- language_policy: OPEN, LOCKED.
- Restrição ck_exercise_lists_locked_language_consistency: (language_policy = 'LOCKED') = (locked_language_id IS NOT NULL).
- Índice ix_exercise_lists_teacher_id sobre teacher_id.

exercises

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
teacher_id	INTEGER	Não	FK: users.id
title	VARCHAR	Não	–

Atributo	Tipo	Nulo	Chaves e referências
description	VARCHAR	Não	–
attachments	VARCHAR	Não	–
language_policy	languagepolicy	Não	–
locked_language_id	INTEGER	Sim	FK: languages.id; exclusão: RESTRICT
created_at	TIMESTAMP WITHOUT TIME ZONE	Não	–
updated_at	TIMESTAMP WITHOUT TIME ZONE	Não	–

- language_policy: OPEN, LOCKED.
- Restrição ck_exercises_locked_language_consistency: (language_policy = 'LOCKED') = (locked_language_id IS NOT NULL).
- Índice ix_exercises_teacher_id sobre teacher_id.

languages

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
owner_id	INTEGER	Não	FK: users.id; exclusão: CASCADE
name	VARCHAR	Não	–
description	VARCHAR	Sim	–
customization	JSONB	Não	–
image_url	VARCHAR	Sim	–
image_query	VARCHAR	Sim	–
preset_id	VARCHAR	Sim	–
cloned_from_id	INTEGER	Sim	FK: languages.id; exclusão: SET NULL
is_public	BOOLEAN	Não	–
published_at	TIMESTAMP WITH TIME ZONE	Sim	–

Atributo	Tipo	Nulo	Chaves e referências
created_at	TIMESTAMP WITHOUT TIME ZONE	Não	–
updated_at	TIMESTAMP WITHOUT TIME ZONE	Não	–

- Unicidade conjunta: owner_id, name.
- Índice ix_languages_owner_id sobre owner_id.
- Índice ix_languages_public_published_at sobre is_public, published_at.

organizations

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
name	VARCHAR	Não	–
created_at	TIMESTAMP WITHOUT TIME ZONE	Não	–

submissions

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
exercise_id	INTEGER	Não	FK: exercises.id
exercise_list_id	INTEGER	Não	FK: exercise_lists.id
class_id	INTEGER	Não	FK: classes.id
student_id	INTEGER	Não	FK: users.id
code_snapshot	VARCHAR	Não	–
language_snapshot	JSONB	Não	–
test_case_results	JSONB	Não	–
status	submissionstatus	Não	–
score	FLOAT	Sim	–

Atributo	Tipo	Nulo	Chaves e referências
teacher_feedback	VARCHAR	Sim	–
submitted_at	TIMESTAMP WITHOUT TIME ZONE	Não	–

- status: PENDING, SUBMITTED, GRADED, LATE.
- Índice ix_submissions_exercise_list_class sobre exercise_list_id, class_id.
- Índice ix_submissions_student_id sobre student_id.

test_cases

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
exercise_id	INTEGER	Não	FK: exercises.id; exclusão: CASCADE
label	VARCHAR	Não	–
input	VARCHAR	Não	–
expected_output	VARCHAR	Não	–
order_index	INTEGER	Não	–

users

Atributo	Tipo	Nulo	Chaves e referências
id	INTEGER	Não	PK
organization_id	INTEGER	Sim	FK: organizations.id
role	userrole	Não	–
email	VARCHAR	Não	UQ
password	VARCHAR	Não	–
name	VARCHAR	Não	–
avatar_url	VARCHAR	Sim	–
bio	VARCHAR	Sim	–
active_language_id	INTEGER	Sim	FK: languages.id; exclusão: SET NULL

- `role`: ADMIN, TEACHER, STUDENT, COMMUNITY, SYSTEM.